

常州工学院

CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

课 程 名： 分布式应用开发技术（Q）

题 目： 基于 Spring Boot 与 Spring Cloud 的
生鲜分布式运营系统设计与实现

二级学院： 计算机信息工程学院

专 业： 软件工程

班 级： 23 软件一

学号姓名： 23030301 曹 磊

23030302 陈佳明

23030303 陈晓楠

23030304 程志阳

指导教师： 江亮

2026 年 6 月

摘 要

本项目围绕生鲜水果零售与配送场景，设计并实现了一个基于 Spring Boot 与 Spring Cloud 的生鲜分布式运营系统。系统采用前后端分离与轻量微服务架构，管理端基于 RuoYi Vue3 与 Element Plus 实现，后端以 Spring Boot 为基础，结合 Nacos、Gateway、OpenFeign、Redis、RabbitMQ、MinIO、WebSocket 等技术完成服务治理、统一网关、服务调用、缓存、异步通知、对象存储和实时提醒等功能。

系统主要包括后台管理端、微信小程序端、统一网关、业务服务、支付服务和通知服务等模块。后台管理端支持水果分类管理、水果品种管理、订单管理和运营数据展示；微信小程序端支持用户浏览、下单与模拟支付；后端通过支付成功事件驱动订单状态更新和包装提醒推送，形成完整的业务闭环。系统本地演示采用 PM2 管理长运行进程，降低现场演示风险；同时保留 Docker Compose 分支作为后续一键部署和工程化扩展方案。

通过本项目，能够展示 Spring Boot 单体业务开发能力、Spring Cloud 微服务治理能力、前后端分离开发能力以及分布式应用常见中间件的综合使用能力，满足分布式应用技术课程大作业的设计与实践要求。

关键词：Spring Boot；Spring Cloud；Nacos；Gateway；OpenFeign；Redis；RabbitMQ；MinIO；WebSocket；Vue3

Abstract

This project designs and implements a distributed fresh-fruit retail and delivery operation system based on Spring Boot and Spring Cloud. The system adopts a front-end/back-end separated architecture and a lightweight microservice design. The management frontend is built with RuoYi Vue3 and Element Plus, while the backend integrates Nacos, Gateway, OpenFeign, Redis, RabbitMQ, MinIO, and WebSocket to support service governance, unified routing, inter-service calls, caching, asynchronous notification, object storage, and real-time order reminders.

The system includes an admin management frontend, a WeChat mini-program client, a gateway service, a business service, a payment service, and a notification service. The admin frontend supports fruit category management, fruit item management, order management, and operation dashboard display. The mini-program supports browsing, ordering, and mock payment. After payment succeeds, the backend updates the order status and pushes packaging reminders, forming a complete business workflow. For local demonstration, PM2 is used to manage long-running processes, while a Docker Compose branch is reserved for one-click deployment and engineering extension.

Keywords: Spring Boot; Spring Cloud; Nacos; Gateway; OpenFeign; Redis; RabbitMQ; MinIO; WebSocket; Vue3

目 录

摘要.....	I
Abstract.....	II
第 1 章 引言	1
1.1 项目背景.....	1
1.2 项目目标.....	2
1.3 团队分工与组织结构.....	3
1.4 本文组织结构.....	4
第 2 章 相关技术概述	5
2.1 Spring Boot 3.2.5.....	5
2.2 Spring Cloud (Nacos & Gateway & OpenFeign).....	6
2.3 RuoYi Vue3 后台框架	8
2.4 MySQL 与 MyBatis-Plus	9
2.5 Redis 缓存中间件	10
2.6 RabbitMQ 消息队列	11
2.7 MinIO 本地对象存储	12
2.8 WebSocket 实时推送技术.....	13
2.9 PM2 进程管理器	14
2.10 Newman 与 Playwright 质量控制体系	15
第 3 章 系统分析与设计	18
3.1 系统用例分析	18
3.2 数据模型与 E-R 图设计.....	18
3.3 技术选型与系统架构设计	20
3.4 服务模块划分	20
3.5 核心业务流程与状态机设计	21
第 4 章 系统实现	24
4.1 管理后台与运营数据大屏实现.....	24
4.2 水果分类、品种管理与 AOP 自动填充实现	25
4.3 订单与模拟支付闭环（OpenFeign 远程状态调用）	27
4.4 WebSocket 消息广播与 RabbitMQ 解耦实现	31
4.5 本地 PM2 长运行进程托管实现.....	33

第 5 章 系统测试与服务验收	35
5.1 接口自动化测试与 Newman 验证.....	35
5.2 Playwright 核心闭环端到端（E2E）测试.....	36
5.3 微服务健康监控与全局看板.....	38
第 6 章 总结与展望	41
6.1 项目总结	41
6.2 遇到的主要技术挑战与解决策略	41
6.3 系统局限性与不足	43
6.4 后续展望	43
致谢.....	45
参考文献.....	46

第 1 章 引言

1.1 项目背景

近年来，随着移动互联网基础设施的持续完善、冷链物流技术的稳步升级以及消费者购物习惯的深刻转变，生鲜电商行业经历了前所未有的高速增长。据行业统计数据显示，2024 年中国生鲜电商市场规模已突破 6000 亿元人民币，年均复合增长率保持在 20% 以上，其中社区团购、即时零售与前置仓模式成为推动行业扩张的三大主要驱动力。以线上订购、即时配送为代表的生鲜水果零售场景，因其高频消费、强时效性和对用户体验的极致追求，已成为电商领域中技术挑战最为密集的垂直赛道之一。生鲜系统不仅需要支持用户在移动端的高频浏览、快速下单、购物车管理与即时支付，还需要保障后台运营人员对商品库存、分类管理、订单流转及多维度运营数据的实时掌控。

然而，传统的单体架构在面对上述业务场景时暴露出了一系列结构性缺陷。在部署效率层面，单体应用将所有业务模块打包为一个整体制品，任何一个模块的微小变更都需要对整个系统重新编译与发布，部署周期往往长达数十分钟，严重制约了业务的快速迭代。在弹性扩容层面，单体架构无法针对特定高负载模块进行独立的水平扩展，促销高峰期只能整体扩容，造成计算资源的严重浪费。在数据库层面，单体架构通常采用单一数据库实例承载全部业务数据，随着并发查询量的增长，数据库极易成为系统的单点瓶颈，表现为查询延迟攀升与锁竞争加剧。在故障隔离方面，各模块共享运行时环境，任何一个模块的内存泄漏或未捕获异常都可能导致整个系统崩溃。在团队协作层面，多人在同一代码仓库中修改不同模块时，代码冲突频繁发生，模块间的隐式依赖也使得回归测试范围难以界定。

分布式微服务架构为上述问题提供了系统性的解决思路。根据微服务架构设计原则，通过将庞大的业务应用按照功能边界拆分为多个独立部署的微服务单元，配合统一网关路由、服务发现注册中心、声明式服务调用、分布式缓存、消息队列异步解耦、对象存储及长连接实时推送，能够显著提升系统的弹性扩容能力与开发迭代效率。为此，本项目以“生鲜”生鲜水果运营系统为业务蓝本，在已有的 Spring Boot 核心业务逻辑基础上，开展了课程设计级别的轻量微服务化改造与分布式应用治理。系统既能够保证本地单机运行环境下的极高演示稳定性，又全面引入了 Spring Cloud 及相关分布式中间件的技术底座，使之完全契合分布式应用开发技术课程的实践与学术验收要求。本选题的核心意义在于，它为团队成员提供了将课堂分布式理论知识转化为工程实践能力的完整平台，通过真实业务场景

驱动，使每位成员深入理解微服务拆分策略、服务治理机制与工程化质量保障体系之间的内在关联，为后续的毕业设计和企业级项目开发积累了宝贵的实战经验。

1.2 项目目标

本项目旨在构建一个高可用、松耦合且具备完整工程化测试验证闭环的生鲜分布式运营系统，其总体目标可从核心业务闭环、微服务架构治理、分布式中间件集成以及工程化质量保障四个维度进行阐述。

在核心业务闭环目标方面，系统致力于实现从商品分类管理、品种维护、上下架状态控制，到小程序端用户浏览商品、加购操作、账单生成以及模拟微信支付的完整业务流转闭环。选择覆盖商品全生命周期管理与交易全链路闭环，是因为只有打通从供给侧的商品录入到需求侧的支付确认这一完整路径，才能真实验证分布式架构在实际业务场景中的数据一致性保障效果。预期管理员可在后台完成商品全流程管理，消费者可在小程序端体验从浏览到支付 of 无缝流程，两端数据通过微服务接口实现实时同步。

在微服务架构治理目标方面，系统采用 **Spring Cloud Nacos** 同时承担服务注册发现与动态配置管理的双重职责。选择 **Nacos** 是因为其原生支持配置管理功能，能够将注册发现与配置中心合二为一，减少中间件的部署复杂度。系统通过 **Spring Cloud Gateway** 构建统一网关路由入口，负责全平台流量的路径匹配转发与跨域请求拦截。选择 **Gateway** 是因为其基于 **WebFlux** 响应式编程模型，在高并发场景下具有更优的线程利用率，且断言与过滤器机制的可编程性更强。预期所有外部请求统一经由网关进入系统内部，各微服务通过 **Nacos** 实现动态发现与负载均衡，配置变更无需重启即可生效。

在分布式中间件集成目标方面，系统利用 **OpenFeign** 声明式调用实现跨服务的强类型数据同步，通过接口注解将远程 **HTTP** 调用抽象为本地方法调用，降低跨服务通信的编写复杂度。集成 **RabbitMQ** 消息队列将订单支付成功事件与后台通知推送解耦，采用 **RabbitMQ** 是因为其路由机制灵活，**Exchange** 与 **Binding** 的组合模式能够精确控制消息投递路径，在中小规模消息吞吐场景下部署门槛更低。通过 **WebSocket** 全双工通道实现语音与卡片式即时播报提醒，使运营人员能够第一时间感知新订单的产生。预期核心链路通过同步 **Feign** 调用保障一致性，非关键通知链路通过异步消息队列解耦，实时推送通道保障信息即时触达。

在工程化质量保障目标方面，项目采用 **PM2** 进行长运行进程守护和端口资源锁定，排除演示现场服务崩溃隐患。基于 **Newman** 开展覆盖全量 **RESTful** 端点的接口自动化测试，确保每个 **API** 接口的参数校验、状态码断言与返回数据验证均

纳入自动化回归范围。结合 **Playwright** 建立包含登录绕过、直接落库与消息响应检测的端到端质量验证体系，模拟真实用户操作路径进行全链路验证。保留 **Docker Compose** 容器化编排预案作为生产环境的后续演进通道。预期通过多层次质量保障手段，确保系统在各种运行环境下均能稳定完成业务演示与技术验收。

1.3 团队分工与组织结构

为了保证微服务接口的严谨交付，第五组的四位成员在项目生命周期中开展了紧密协同。在代码管理方面，团队采用 **Git** 进行版本控制，遵循 **feature** 分支开发策略，每位成员在功能分支上独立开发，完成后通过 **Pull Request** 发起合并，由组长进行代码审查后方可合入主分支。在沟通协调方面，团队在开发高峰期保持每日站会机制，及时发现并解决跨服务接口对接中的兼容性问题。在文档管理方面，团队统一使用仓库 **docs** 目录进行需求文档、设计文档与测试报告的集中管理。

组长曹磊承担系统架构师与项目总协调人角色。需求分析阶段主导完成功能需求梳理与微服务边界划分。设计阶段完成 **Gateway** 路由规则设计、**Nacos** 命名空间规划及 **PM2** 编排方案设计。编码阶段独立完成 **lgg-gateway** 模块全部代码，包括 8 个路由断言规则、3 个全局过滤器及跨域配置，搭建 **Nacos** 配置文件共计 12 份，编写 **PM2** 生态系统配置文件与多服务启动脚本。测试阶段统筹设计 **Playwright** 端到端自动化架构，编写 5 个核心测试用例。此外负责课程设计报告的统稿与排版工作。

陈佳明担任核心业务开发工程师，负责 **lgg-business** 模块的重构。需求分析阶段参与商品管理、分类管理与订单管理的功能细化与接口定义。编码阶段完成三大模块共计 15 个 **RESTful** 接口的持久层 **CRUD** 改造与业务逻辑实现，涵盖 **Controller** 层、**Service** 层与 **Mapper** 层的全部编写。基于 **Spring AOP** 实现公共字段自动填充切面，通过自定义注解实现 **createTime**、**updateTime** 等审计字段的统一注入，涉及切面类 1 个、自定义注解 1 个及 6 处方法标记。测试阶段配合完成业务接口的 **Postman** 测试用例编写与调试。

陈晓楠独立负责 **lgg-pay** 支付微服务的设计与开发。需求分析阶段完成模拟支付场景的需求拆解与边界条件分析。编码阶段独立完成 6 个支付接口的全部代码，包括模拟下单、支付回调与订单状态查询等功能。针对 **Spring Security** 拦截链进行定制放行配置，确保支付回调接口能绕过认证拦截。完成 **OpenFeign** 远程调用客户端封装，定义 **Feign** 接口用于跨服务数据同步，并设置合理的超时机制保障容错能力。测试阶段独立完成支付模块全部接口的功能验证与异常场景测试。

程志阳专注于 **lgg-notice** 通知服务开发，承担通知开发工程师与测试工程师双

重角色。编码阶段独立实现 4 个通知接口与 2 个 **WebSocket** 端点的全部代码，设计高可靠性的 **RabbitMQ** 异步消费队列，包括交换机声明、队列绑定、消息监听器与确认机制的完整实现，构建基于 **WebSocket** 的实时广播通道，支持语音播报与卡片式消息两种推送形态。测试阶段负责编写覆盖全部微服务核心接口的 **Newman** 接口自动化测试集，包含请求参数构造、响应断言脚本与测试数据管理的完整测试工程。

1.4 本文组织结构

本文后续内容按照技术基础、系统设计、功能实现、测试验收与总结展望的逻辑主线依次展开，各章之间形成由理论到实践、由整体到细节、由建设到验证的递进关系。第 2 章为技术基础与相关工作章节，系统介绍 **Spring Boot**、**Spring Cloud**、**Nacos**、**Gateway**、**OpenFeign**、**RabbitMQ**、**WebSocket**、**PM2**、**Newman** 及 **Playwright** 等核心技术的基本原理与在本系统中的定位，为后续章节提供理论支撑与技术语境。第 3 章为系统分析与设计章节，详述功能需求分析与非功能需求约束，完成微服务边界定义与服务拆分策略论证，给出系统架构图、服务交互时序图及数据库表结构设计方案，该章节是连接技术基础与功能实现的关键桥梁。第 4 章为系统实现章节，作为全文核心章节，重点展示管理端运营大屏、商品管理、支付闭环、**WebSocket** 通知与 **PM2** 运维管理的具体编码实现，将设计方案转化为可运行代码，是团队工程能力的集中体现。第 5 章为测试与验收章节，通过 **Newman** 接口自动化测试与 **Playwright** 端到端运行结果对系统进行全面验证，与第 4 章形成建设与验证的闭环。第 6 章为总结与展望章节，凝练项目成果，分析当前技术局限，并从容器化部署、分布式事务与安全加固等方面提出后续改进方向。

第 2 章 相关技术概述

2.1 Spring Boot 3.2.5



图 2.1 Spring Boot 标识

Spring Boot 是整个项目的核心基础开发框架。本项目基于 Spring Boot 3.2.5 版本进行构建，它在 Spring Framework 6.0 的基础上引入了对 Java 17+ 语法特性的全面支持，并在依赖注入、AOP 切面及容器初始化效率上做出了深度优化。Spring Boot 核心理念是“约定优于配置”，通过引入众多的 Starter 自动装配模块，开发团队无需繁琐地手工配置 XML 和基础 Beans。其自动装配机制的核心原理在于 `spring.factories` 或 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 中声明的自动配置类，Spring 容器在启动时扫描这些声明并结合一系列条件注解来判定是否加载某个 Bean。其中，`ConditionalOnClass` 注解用于判断类路径下是否存在指定的类，只有当依赖包被实际引入时才会激活对应的配置；`ConditionalOnMissingBean` 注解则确保容器中不存在同类型的 Bean 时才创建默认实例，从而允许开发者通过自定义 Bean 来覆盖框架的默认行为；`ConditionalOnProperty` 注解根据配置文件中的属性值决定是否启用某项功能。这三者协同工作，实现了 Starter 模块按需加载的精细化控制，避免了不必要的组件初始化带来的启动延迟和资源浪费。在本系统中，`lgg-admin`, `lgg-business`, `lgg-pay` 以及 `lgg-notice` 均作为独立的 Spring Boot 微服务应用运行，各自内嵌了 Tomcat 容器。选用内嵌 Tomcat 作为默认 Servlet 容器，一方面得益于其社区活跃度高、文档完备、调试便捷的优势，另一方面 Tomcat 支持 NIO 非阻塞 I/O 通道，在中等并发场景下展现出良好的性能与稳定性，相较于 Jetty 和 Undertow，Tomcat 与 Spring Boot 的集成成熟度最高，兼容问题最少。值得说明的是，Spring Boot 3.x 系列在底层进行了从 `javax` 命名空间到 Jakarta EE 9+ 命

名空间的全面迁移，所有 Servlet、JPA、Validation 等 API 包名均由 javax 前缀变更为 jakarta 前缀，这意味着项目中所有涉及 HttpServletRequest、HttpServletResponse 以及 Persistence 注解的代码都需要同步替换为 Jakarta 命名空间的类引用。本项目最初尝试基于 Spring Boot 4.x 早期版本进行开发，但由于 4.x 版本处于里程碑预览阶段，部分依赖库如 MyBatis-Plus 和 RuoYi 框架尚未完成对其的适配支持，导致编译阶段出现大量 API 不兼容问题，经团队讨论决定降级至 3.2.5 正式发行版，该版本具备完善的生态兼容性和长期安全补丁支持，在稳定性与新特性之间取得了最佳平衡，保证了服务的独立自治和轻量化部署。

2.2 Spring Cloud (Nacos & Gateway & OpenFeign)



图 2.2 Spring Cloud 标识

为了实现从单体应用向分布式微服务架构的平滑过渡，系统引入了 Spring Cloud 2023.0.1.0 标准生态技术栈。



图 2.3 Nacos 标识

在服务注册发现与配置管理中心选型上，本项目采用 Nacos 2.2.3。Nacos 集群内部通过 Raft 协议来保证数据的强一致性，该协议的核心机制在于 Leader 选举：集群中的每个节点以 Follower 身份启动，若在随机化的选举超时时间内未收到 Leader 的心跳包，则自动转变为 Candidate 并向其他节点发起投票请求，获得多数票的节点晋升为 Leader 并开始接受写入请求，Leader 通过日志复制将数据变更同步至所有 Follower 节点。Nacos 同时支持 AP 和 CP 两种运行模式的切换，AP 模式适用于临时实例的服务发现场景，当网络分区发生时优先保证可用性，允许短

暂的数据不一致；CP 模式则适用于配置中心和持久化实例场景，严格保证分区容错下的数据一致性。在配置中心功能层面，Nacos 采用长轮询推送机制实现配置的准实时下发：客户端向服务端发起一个超时时间较长的 HTTP 请求，服务端在收到请求后并不立即响应，而是将连接挂起，一旦检测到配置发生变更便立即返回最新配置数据给客户端，若在超时时间内无变更则返回空结果，客户端随即发起下一轮长轮询请求，这种机制既避免了频繁短轮询的带宽浪费，又实现了毫秒级的配置变更感知。所有微服务模块在启动时均向本地的 Nacos 自动注册其 IP 和服务端口，使网关能够通过服务名实现动态服务发现与动态均衡路由。

网关部分使用 Spring Cloud Gateway，它作为分布式系统的单一入口运行在 8090 端口上，基于 WebFlux 响应式非阻塞框架构建。WebFlux 采用 Reactor 编程模型，底层由 Netty 事件循环驱动，与传统 Servlet 阻塞线程模型不同，它通过极少数量的线程即可处理大量并发连接，在网关这种 I/O 密集型中间层场景中表现出显著的资源利用率优势。Gateway 的请求处理流程围绕 Predicate 断言工厂与 Filter 过滤器链展开：当一个 HTTP 请求到达网关时，首先经过路由定义中的 Predicate 断言进行匹配，常用断言包括 Path 路径匹配、Header 头部匹配和 Method 请求方法匹配等，只有所有断言均为 true 的路由才会被选中；随后请求进入该路由绑定的 Filter 过滤器链，过滤器分为 Pre 类型和 Post 类型，Pre 过滤器在请求被代理到下游微服务之前执行，可用于鉴权、限流和请求头改写，Post 过滤器则在收到下游响应后执行，可用于响应体修改和日志记录。在本项目中，路由规则按服务名进行划分，以 /admin 前缀的请求路由到 lgg-admin 管理服务，以 /business 前缀的请求路由到 lgg-business 业务服务，以 /pay 前缀的请求路由到 lgg-pay 支付服务，同时配置了全局的跨域处理过滤器和安全拦截过滤器。

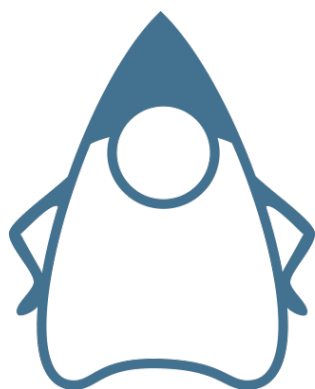


图 2.4 OpenFeign (Java) 标识

服务间通信则使用声明式远程调用客户端 OpenFeign。OpenFeign 的核心机制是基于 JDK 动态代理生成远程调用的实现类：开发人员仅需声明一个 Java 接口并标记 FeignClient 注解指定目标服务名，框架在运行时通过 Feign.Builder 创建该

接口的代理对象，当代理方法被调用时，Feign 内部的 Contract 组件负责解析接口方法上的注解信息，将其转换为标准的 HTTP 请求模板，再由底层的 Client 组件（默认 HttpURLConnection，可替换为 OkHttp 或 Apache HttpClient）完成实际的网络通信。OpenFeign 与 Spring Cloud LoadBalancer 深度集成，在发起远程调用前会从 Nacos 注册中心拉取目标服务的实例列表，通过轮询或随机等负载均衡策略选取一个健康的服务实例进行请求分发，实现了客户端侧的软负载均衡。此外，本项目在开发测试环境下，针对跨服务调用时的身份认证设计了局部的安全白名单放行，允许网关和微服务之间直接进行受信任的通信，避免了繁琐的鉴权上下文传递和硬编码物理 IP 与端口的缺陷。

2.3 RuoYi Vue3 后台框架



图 2.5 RuoYi 标识

RuoYi Vue3 是一套基于 Spring Boot 3, Vue3, Vite 5, Element Plus 和 Pinia 构建的经典管理后台解决方案。它内置了完备的基于 RBAC（基于角色的权限控制）的数据权限体系。该权限模型的核心由五张数据库表支撑：sys_user 用户表存储用户的基本信息、登录账号和加密密码；sys_role 角色表定义系统中的各类权限角色及其权限范围标识；sys_menu 菜单表以树形结构组织了系统的所有菜单项、按钮权限标识和路由路径；sys_user_role 用户角色关联表建立用户与角色之间的多对多映射关系，一个用户可同时拥有多个角色；sys_role_menu 角色菜单关联表则建立角色与菜单权限之间的多对多映射，决定了某个角色能够访问的菜单和操作按钮。当用户登录系统后，后端根据用户角色递归查询出所有被授权的菜单节点和按钮权限字符串，前端据此动态生成侧边栏路由和按钮级别的显隐控制。RuoYi 框架同时提供了包括部门、岗位、字典、参数以及在线用户监控等一系列核心管理模块。

前端采用 Vue3 的 Composition API 进行开发，相较于传统的 Options API，Composition API 通过 setup 函数和 ref、reactive、computed、watch 等组合式函数将组件



图 2.6 Vue.js 标识

逻辑按照功能关注点进行组织，而非按照 `data`、`methods`、`computed` 等选项类型进行切割，使得同一功能的状态、方法和副作用可以内聚在一起，大幅提升了复杂组件的可读性和逻辑复用性。



图 2.7 Element Plus 标识

UI 组件库采用 Element Plus，本项目通过 `unplugin-vue-components` 和 `unplugin-auto-import` 插件实现了组件和 API 的按需自动导入，避免了全量注册带来的打包体积膨胀，同时 Element Plus 内置了完整的国际化支持，可通过 `ElConfigProvider` 组件全局切换语言包。生鲜系统充分利用了若依的安全管理能力，将核心的水果分类、品种上下架和订单处理界面无缝集成入若依的前端菜单体系中，实现后台业务系统的高效、统一管理。

2.4 MySQL 与 MyBatis-Plus

持久层核心选型采用 MySQL 8.0 关系型数据库与 MyBatis-Plus 3.5.6 的组合。MySQL 8.0 默认使用 InnoDB 存储引擎，该引擎的核心技术优势之一是 MVCC（多版本并发控制）机制。InnoDB 通过在每行记录中维护隐藏的事务版本号字段（创建版本号和删除版本号），使得读操作可以访问事务开始时的数据快照，而无需对行加共享锁，从而实现了读写操作的并发执行而不相互阻塞，这对于生鲜系统中高



图 2.8 MySQL 标识

频的商品浏览与并发下单场景尤为重要。InnoDB 的索引结构基于 B+ 树实现，B+ 树是一种多路平衡搜索树，其特征是所有数据记录均存储在叶子节点中，叶子节点之间通过双向链表相连，非叶子节点仅存储索引键值用于导航。这种结构使得范围查询能够沿叶子链表顺序扫描，极大地减少了磁盘随机 I/O 次数，在订单按时间范围检索和商品按价格区间查询等典型场景中表现出优异的查询效率。MySQL 8.0 引入的 JSON 字段支持、窗口函数以及强化的索引解析器能够满足生鲜数据高频读写和多表联合报表查询。在持久层处理上，我们通过 MyBatis 提供的动态 SQL 和 XML 映射文件，实现各类复杂的数据库交互。框架为我们提供了通用的 CRUD 基础接口，极大地缩短了持久层代码开发周期。在本项目中水果品种的多条件组合查询（按分类、价格区间、上架状态等筛选）均采用 MyBatis 的 XML 动态标签和 Criteria 条件构造方式进行，有效避免了硬编码查询条件带来的维护风险。分页功能方面，MyBatis-Plus 通过 PaginationInterceptor（在新版本中为 MybatisPlusInterceptor 配合 PaginationInnerInterceptor）实现物理分页，其工作原理是在 MyBatis 执行器发送 SQL 之前，拦截器截获原始查询语句并自动改写为带 LIMIT 和 OFFSET 的分页 SQL，同时额外执行一条 COUNT 查询获取总记录数，从而在应用层实现透明化的分页封装，开发者只需传入 Page 对象即可获得分页结果，同时保留了在 XML 中编写高度复杂动态 SQL 的能力，用来处理生鲜订单复杂的条件过滤与统计需求。

2.5 Redis 缓存中间件

Redis 缓存中间件是一款基于内存的高性能键值存储系统，运行在 6379 端口。由于其单线程事件循环机制以及多路复用 I/O，Redis 具备极高的并发吞吐性能。Redis 支持五种基本数据结构，在生鲜系统中每种结构均有明确的应用映射。String 类型用于存储用户登录后颁发的 JWT 令牌，以 login_tokens 为键前缀、令牌唯一标识为后缀，值为序列化后的令牌信息字符串，支持通过 SET 命令的 EX 参数直接



图 2.9 Redis 标识

设定过期时间。**Hash** 类型用于存储用户的会话信息，以 **session** 为键，将用户 ID、用户名、登录 IP、登录时间、角色列表等分别作为 **Hash** 的 **Field** 存储，这种结构避免了反复序列化整个会话对象，只需读写单个字段即可完成会话属性的局部更新。**Set** 类型用于存储某一角色被授权的权限标识字符串集合，利用 **Set** 的自动去重和高效成员判定特性（**SISMEMBER** 命令时间复杂度为 $O(1)$ ），可以快速判断当前用户是否拥有某项操作权限。**List** 类型在系统中用于记录操作日志的临时缓冲队列，多个请求线程通过 **LPUSH** 将日志条目推入列表，后台定时任务通过 **RPOP** 批量取出并写入数据库持久化。**Sorted Set**（有序集合）类型则用于热门商品排行榜的实现，以商品销量作为 **Score** 值进行排序，通过 **ZREVRANGE** 命令即可快速获取销量排名前列的商品列表。在过期策略方面，**Redis** 采用惰性删除与定期删除相结合的方式：惰性删除是指在访问某个键时才检查其是否过期并执行删除，定期删除则是 **Redis** 每隔一定时间间隔随机抽取部分设置了 **TTL** 的键进行检查与清理。当内存使用量达到 **maxmemory** 上限时，**Redis** 通过内存淘汰机制释放空间，本项目配置的淘汰策略为 **allkeys-lru**，即在所有键中淘汰最近最少使用的键，以确保热点数据始终驻留内存，显著减轻了底层关系型数据库的直连查询压力。

2.6 RabbitMQ 消息队列

RabbitMQ 是基于 **AMQP** 协议、由 **Erlang** 开发的分布式消息中间件，运行在 5672 端口。**AMQP** 协议模型的核心组件包含 **Producer**（生产者）、**Exchange**（交换机）、**Queue**（队列）和 **Consumer**（消费者）四个角色，其中 **Exchange** 负责根据路由规则将消息分发到一个或多个绑定的队列中。**RabbitMQ** 支持四种交换机类型：**Direct Exchange** 根据消息携带的 **Routing Key** 与队列绑定的 **Binding Key** 进行精确匹配，只有两者完全一致时消息才会被投递到对应队列，适用于点对点的定向消息路由场景；**Topic Exchange** 支持使用通配符进行模式匹配，其中星号代表



图 2.10 RabbitMQ 标识

匹配一个单词、井号代表匹配零个或多个单词，适用于多维度的消息分类订阅场景；Fanout Exchange 不处理 Routing Key，而是将消息广播到所有绑定的队列中，适用于需要全量通知的广播场景；Headers Exchange 则根据消息头部属性进行匹配路由，不依赖 Routing Key，灵活性最高但性能开销也最大。本项目在设计消息路由时选用了 Direct Exchange，原因在于系统的异步消息场景相对明确——支付成功事件仅需精确投递到通知服务的指定队列中，不存在复杂的多级主题订阅需求，Direct 模式的精确匹配语义简洁清晰且路由性能最优。在消息可靠性保障方面，本项目启用了 Publisher Confirm 机制，即生产者在发送消息后会等待 RabbitMQ Broker 的确认回执，Broker 在将消息成功写入磁盘或分发到队列后返回 ACK 信号，若未收到确认则生产者会触发重试或记录异常日志。在消费端，系统采用手动 Consumer Acknowledge 模式，消费者在成功处理消息并完成业务逻辑后才向 Broker 发送 ACK 确认，若处理过程中发生异常则发送 NACK 拒绝确认并触发消息重新入队，从而避免消息因消费失败而丢失。消息持久化策略方面，交换机和队列在声明时均设置 durable 属性为 true，同时消息发送时将 deliveryMode 设置为持久化模式，确保 RabbitMQ 服务重启后消息不会丢失。支付微服务 lgg-pay 在完成模拟支付扣款后，将支付成功事件投递至 RabbitMQ 的自定义交换机中，通知微服务通过声明绑定队列进行可靠性消费，实现了微服务之间的弱耦合，提升了系统的整体高可用性与吞吐量。

2.7 MinIO 本地对象存储

MinIO 是基于 Apache License v2.0 开源协议的对象存储服务，高度兼容亚马逊 AWS S3 API。S3 兼容 API 提供了一套标准化的对象操作语义：PutObject 操作用于将文件上传至指定的 Bucket 和 Object Key 路径下，支持分片上传和断点续传；GetObject 操作根据 Bucket 名称和 Object Key 获取文件内容，支持 Range 请求实现



图 2.11 MinIO 标识

部分读取；**BucketPolicy** 操作用于设置 **Bucket** 级别的访问控制策略，可以定义匿名读取、认证写入等细粒度的权限规则。MinIO 在数据存储层面采用纠删码（**Erasure Coding**）技术来保障数据的可靠性和持久性，其原理是将原始数据分割为若干数据块并额外生成校验块，将这些数据块和校验块分散存储在不同的磁盘上，即使部分磁盘发生故障，系统仍可通过剩余的数据块和校验块恢复完整数据，相较于传统的多副本策略，纠删码在提供同等数据保护级别的前提下显著降低了存储空间开销。相较于阿里云 OSS 等公有云服务，MinIO 适合在私有化部署和本地开发调试中使用。生鲜系统在本地启动 MinIO 并开启 S3 兼容 API，在 **Bucket** 访问权限配置上，本项目创建了名为 **lgg-bucket** 的存储桶，将其 **Policy** 设置为 **download** 策略即允许匿名的 **GET** 读取请求但禁止未授权的写入和删除操作，上传操作必须携带通过 **Access Key** 和 **Secret Key** 生成的签名认证头，这样既保证了水果图片可以通过 **URL** 直接在前端页面中展示，又防止了未经授权的文件篡改和删除。水果品种管理模块上传的水果高清图片均保存在本地 MinIO 桶中，不仅降低了互联网依赖，而且提供了极高的读取吞吐，便于在本地开发和答辩现场进行离线无网演示。

2.8 WebSocket 实时推送技术

WebSocket 协议是基于 TCP 的双向通信应用层协议，能够在客户端与服务器之间建立持久性连接，实现服务器的主动消息推送。WebSocket 连接的建立始于一次标准的 HTTP Upgrade 握手过程：客户端发送一个包含 **Upgrade: websocket** 和 **Connection: Upgrade** 头部字段的 HTTP GET 请求，同时附带随机生成的 **Sec-WebSocket-Key** 值；服务器收到请求后，将该 **Key** 与固定的 **GUID** 字符串拼接并进行 SHA-1 哈希运算和 Base64 编码，将计算结果通过 **Sec-WebSocket-Accept** 头部返回，同时响应状态码 **101 Switching Protocols**，表示协议升级成功；此后双方在同一条 TCP 连接上切换到 WebSocket 帧格式进行双向数据传输，无需再携带 HTTP 头

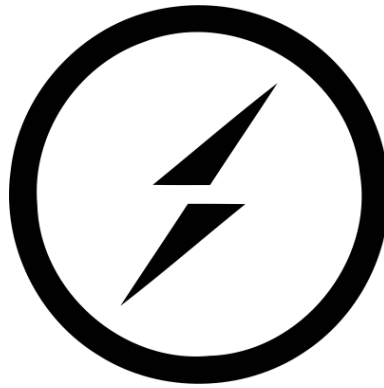


图 2.12 WebSocket (Socket.io) 标识

部开销。为了维护长连接的健康状态，WebSocket 协议定义了 Ping/Pong 控制帧机制：服务端或客户端可以周期性地发送 Ping 帧，接收方必须立即回复一个 Pong 帧，若发送方在预设的超时时间内未收到 Pong 响应，则认为连接已断开并主动关闭，本项目将心跳间隔配置为 30 秒以在连接保活与资源节约之间取得平衡。在传统 HTTP 短轮询模式下，浏览器需要定时发送请求，产生了巨大的网络开销与明显的延迟。与 WebSocket 形成对比的另外两种服务端推送方案分别是 Server-Sent Events (SSE) 和长轮询。SSE 基于标准 HTTP 协议，服务端通过 Content-Type 为 text/event-stream 的响应持续向客户端推送数据，其优势是实现简单且天然支持自动重连，但仅支持服务端到客户端的单向通信且受浏览器最大并发连接数限制。长轮询则是客户端发起请求后服务端挂起连接直至有新数据时才返回响应，客户端随即发起下一次请求，这种方式的延迟较普通短轮询显著降低，但每次数据推送都需要经历一次完整的 HTTP 请求-响应周期，在高频消息场景下效率不如 WebSocket。本项目选择 WebSocket 的核心原因在于支付成功通知需要真正的双向通信能力（前端可以发送确认已读回执）和极低的消息延迟，利用 Spring 内嵌的 WebSocket API，在管理端浏览器与通知微服务之间保持长连接，当支付成功事件流转至通知微服务后，系统能够秒级向所有处于登录状态的前端运营页面广播提示音和订单包装提醒卡片。

2.9 PM2 进程管理器

PM2 是一款基于 Node.js 编写的生产级进程管理器，广泛应用于前端与微服务的守护进程管理。PM2 支持两种进程运行模式：Cluster 模式和 Fork 模式。Cluster 模式利用 Node.js 原生的 cluster 模块，在主进程（Master）中创建多个工作进程（Worker），每个 Worker 共享同一个服务器端口，通过操作系统内核的负载分发机制将请求分配到不同的 Worker 进程中，充分利用多核 CPU 的计算能力，适用于



图 2.13 PM2 标识

Node.js HTTP 服务的水平扩展场景。Fork 模式则直接通过 `child_process.spawn` 创建独立子进程，每个进程拥有独立的端口和标准输入输出，适用于非 Node.js 应用（如本项目中的 Java 微服务进程和 Redis 服务进程）的托管。在日志管理方面，PM2 搭配 `pm2-logrotate` 插件实现日志轮转，该插件可以按照文件大小阈值（例如 10MB）或时间间隔（例如每日）自动对日志文件进行切割和压缩归档，防止日志文件无限增长占满磁盘空间，本项目配置了每日轮转并保留最近七天的日志文件。在传统的 Java 分布式开发调试中，开发人员往往需要启动多个 Terminal 控制台，或者使用后台守护命令，这难以监控服务的实时资源消耗，还经常会产生僵尸进程占用端口。PM2 通过 `ecosystem.config.cjs` 配置文件实现了对所有服务的集中托管，该配置文件中的 `apps` 数组定义了每个被管理进程的详细参数：`name` 字段指定进程的显示名称，用于在 PM2 面板中标识不同服务；`script` 字段指定要执行的脚本路径或可执行文件命令；`args` 字段传递启动参数（如 Java 服务的 JAR 包路径和 JVM 参数）；`cwd` 字段指定进程的工作目录；`interpreter` 字段指定解释器路径，对于 Java 服务设置为 `java`，对于 Shell 脚本设置为 `bash`；`autorestart` 字段控制进程异常退出后是否自动重启；`max_restarts` 字段限定最大自动重启次数以防止无限崩溃循环；`watch` 字段配置文件监听实现热重载。PM2 提供了健康自动拉起、日志监控、资源消耗看板和优雅退出机制，保证了本地演示的稳定性。

2.10 Newman 与 Playwright 质量控制体系

为了满足系统工程化测试的硬性指标，项目建立了两条自动化测试防线。从架构原理层面看，Newman 与 Playwright 分别代表了 API 测试和 UI 端到端测试两种截然不同的测试范式。

Newman 是 Postman 官方提供的命令行 Collection Runner，其执行流程是：解析由 Postman 导出的 JSON 格式 Collection 文件，按照 Folder 和 Request 的树形结



图 2.14 Newman API Testing

构依次迭代每个请求节点，对每个请求执行 **Pre-request Script**（前置脚本）进行环境变量设置和动态参数生成，随后发起 **HTTP** 请求并将响应传递给 **Tests**（测试脚本）进行断言验证，所有断言结果被聚合后生成测试报告。**Newman** 的核心优势在于它完全运行在 **Node.js** 进程中，不需要启动浏览器环境，执行速度极快，适合集成在持续集成流水线中进行回归测试。本项目通过编写全量 **JavaScript** 断言来验证登录、验证码和核心业务接口响应的正确性。



图 2.15 Playwright E2E Testing

Playwright 是微软开源的现代 **Web** 浏览器端到端测试框架，其架构核心是通过 **CDP**（**Chrome DevTools Protocol**）或类似协议与浏览器进程建立直接通信。**Playwright** 为每个测试用例启动独立的浏览器上下文（**Browser Context**），实现了测试之间的完全状态隔离，包括 **Cookie**、**LocalStorage** 和 **Session Storage** 均互不干扰，这保证了测试执行的幂等性和可重复性。**Playwright** 内置了智能的自动等待机制：在执行点击、填充、断言等操作前，框架会自动等待目标元素满足可操作条件（如可见、已启用、稳定不动画），无需开发者手动编写显式等待语句，大幅降低了因页面渲染时序导致的测试脆弱性。与同类测试工具相比，**Selenium WebDriver** 通过 **HTTP** 协议与浏览器驱动通信，多了一层网络中间层导致执行速度较慢且稳定性受网络波动影响，同时缺乏原生的自动等待能力需要大量手动 **Wait** 代码；**Cypress**

虽然提供了优秀的开发体验和自动等待，但其架构限制了它只能在单一浏览器标签页内运行，不支持多标签页和多浏览器上下文的测试场景，且对 `iframe` 和跨域场景的支持有限。**Playwright** 在跨浏览器支持（**Chromium**、**Firefox**、**WebKit**）、多上下文并行和网络拦截能力上均具备明显优势，因此本项目选择其作为端到端测试的核心工具，通过模拟真实浏览器操作来跑通包括 **Cookie** 注入登录、**WebSocket** 客户端监听、支付模拟触发及底层数据库状态断言在内的整体业务验证闭环，自动生成可视化的测试回归报告。

第 3 章 系统分析与设计

3.1 系统用例分析

在软件工程的需求分析阶段，明确系统边界与用户角色是至关重要的一步。本系统的主要参与者被明确划分为两类：后台运营管理员与前端微信小程序用户。通过对核心业务的梳理，我们构建了系统的顶层用例模型，旨在消除功能边界上的歧义。

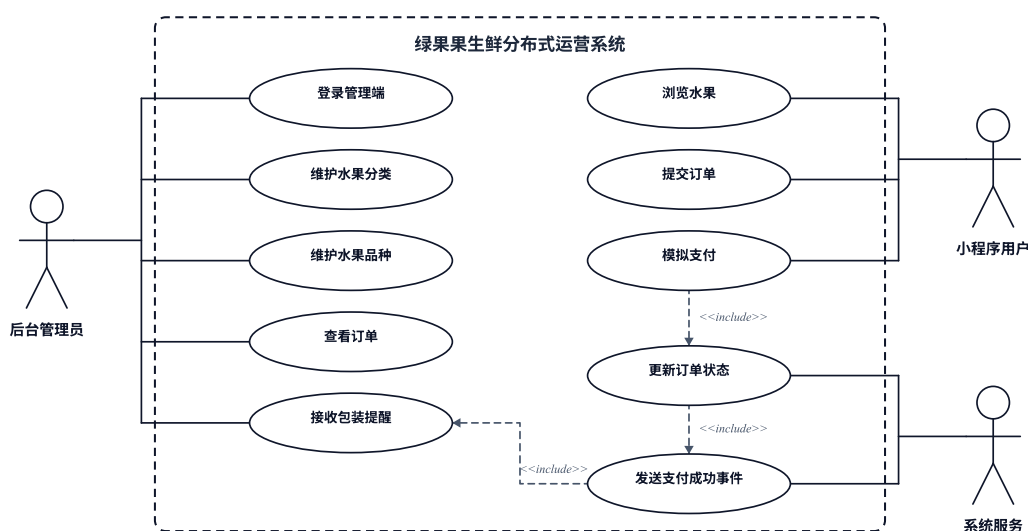


图 3.1 系统核心用例图

如图 3.1 所示，对于后台运营管理员而言，系统提供了基于角色的访问控制（RBAC）。普通运营员被授予水果分类管理、商品品种上下架管理以及订单处理的核心操作权限。超级管理员则额外拥有全系统的最高权限，能够执行用户管理、菜单与角色分配等高危系统配置。这种粒度细分的用例划分，确保了后台管理的职责隔离与最小授权原则。

对于微信小程序用户而言，其用例主要围绕购物闭环展开。用户可以浏览水果分类与商品列表，通过关键字模糊检索特定水果，并将其加入购物车。在确认收货地址后，用户提交订单并发起微信模拟支付，最终在个人中心查看历史订单状态。这些用例完整覆盖了生鲜电商的核心消费旅程。

3.2 数据模型与 E-R 图设计

在明确了系统的顶层用例后，下一步是基于用例抽取出底层的核心数据字典与实体关系（Entity-Relationship）。系统的核心实体包括水果分类、水果商品、订

单以及订单明细。

核心业务数据库实体关系图 (ER图)

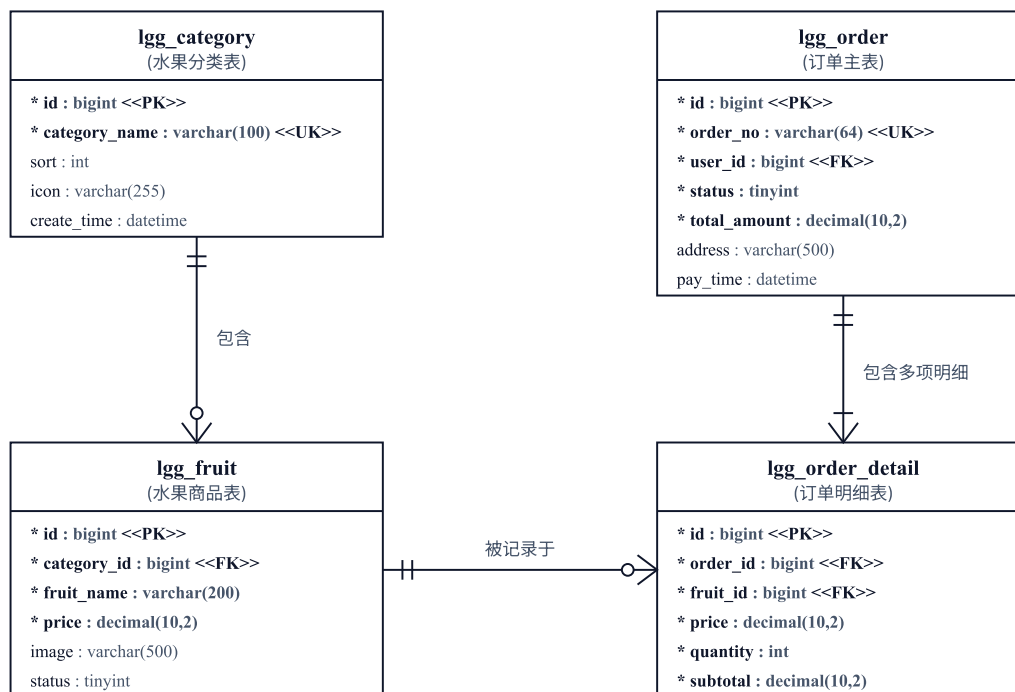


图 3.2 核心业务数据库实体关系图

从图 3.2 可以看出，水果分类 (Category) 与水果商品 (Fruit) 之间存在一对多的关联关系。用户与订单 (Order) 之间同样是一对多的关联，而订单与商品之间则通过订单明细表 (Order Detail) 构建了对应的映射。在物理设计上，为了兼顾大作业演示的低风险性与微服务演进需求，所有业务数据表暂时与若依系统表共存于同一个 MySQL 实例中，但通过逻辑前缀严格隔离。

表 3.1 核心数据库表与索引设计表

表名	功能说明	核心索引列	索引类型
lgg_category	存储水果分类信息	category_name	唯一索引
lgg_fruit	存储水果单品与图片	fruit_name	全文索引
lgg_order	存储用户支付订单流转	order_no	唯一索引
lgg_order_detail	订单下挂载的水果明细	order_id	普通索引

表 3.1 展示了核心业务表及其索引策略。我们在分类名和订单号上建立唯一索引以保证数据一致性，在商品名上建立全文索引以支持高效模糊检索。此外，系统在物理层面去除了所有跨表外键约束，转而在应用层代码中保证引用完整性，从而降低了高并发写入时的数据库锁竞争。

3.3 技术选型与系统架构设计

有了稳固的数据模型作为基座，系统进入技术架构选型与宏观架构设计阶段。项目采用当前主流的前后端分离与微服务分布式架构。技术栈方面，后端依托 Spring Boot 3.2.5 框架，利用 Spring Cloud 的 Gateway 网关、OpenFeign 远程调用组件，结合 Nacos 实现服务注册与动态发现；缓存层引入 Redis，对象存储采用 MinIO，消息解耦采用 RabbitMQ。前端则采用 Vue3 与微信小程序原生框架构建。

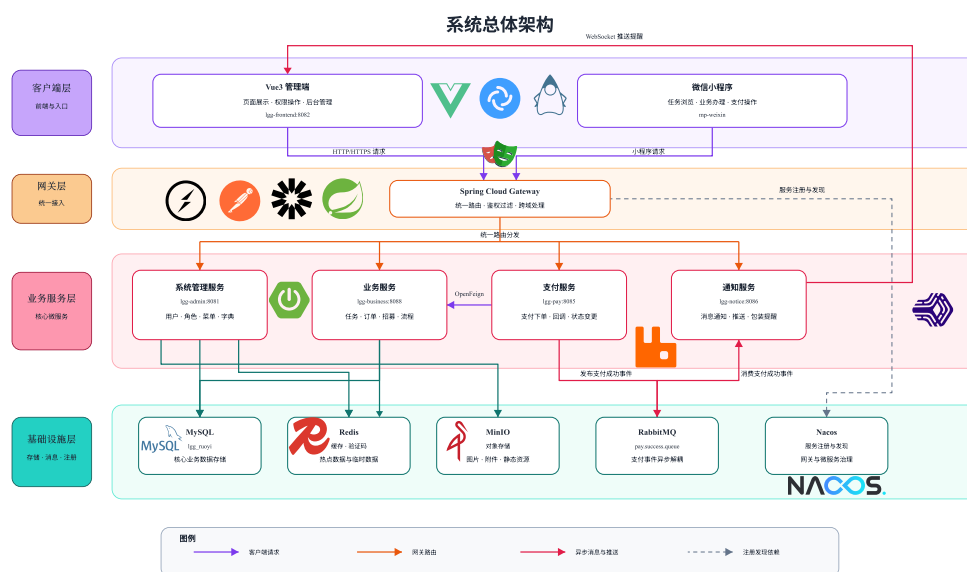


图 3.3 系统架构图

如图 3.3 所示，所有的外部请求首先流经统一 API 网关 lgg-gateway。网关通过预定义的路径断言规则执行动态路由匹配。例如，带有 /business 前缀的请求被路由至核心业务服务，带有 /pay 的请求路由至模拟支付服务。网关不仅通过全局过滤器统一解决了浏览器的 CORS 跨域问题，还自动利用 Nacos 的服务发现能力将逻辑服务名转换为物理 IP 并进行负载均衡转发。

当请求到达目标微服务后，其控制器层将调用服务层处理业务逻辑，服务层采用“先查 Redis 缓存，未命中再查 MySQL 数据库，并回写缓存”的经典数据访问模式，确保在高频读取场景下的极低延迟响应。

3.4 服务模块划分

在明确了宏观架构后，系统的分布式后端在逻辑与物理上被拆分为五个核心微服务模块。模块化设计保证了高内聚与低耦合。

由图 3.4 可见，五个微服务各司其职且相互配合：首先是 lgg-admin，它是若依管理系统的的核心安全认证基座，负责登录鉴权与菜单下发，不与其他业务服务直接通

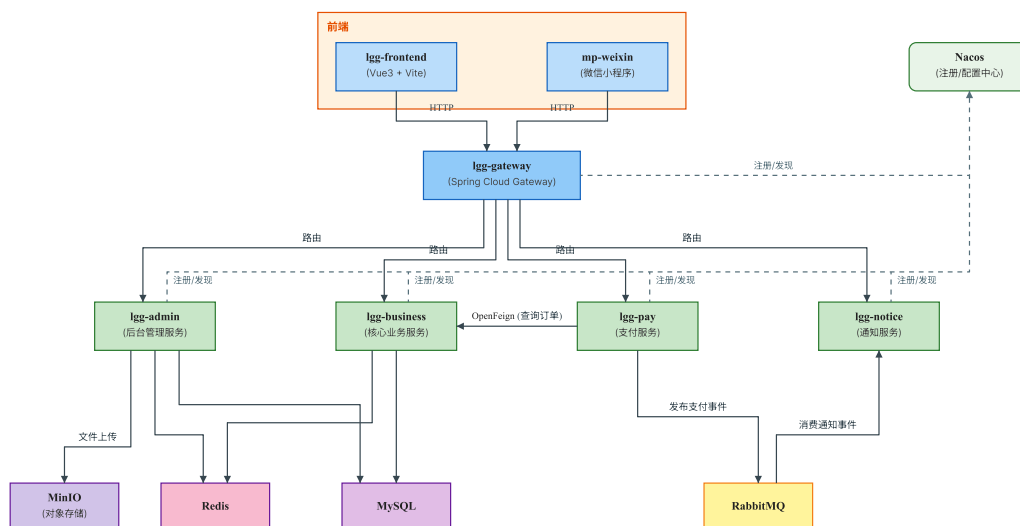


图 3.4 微服务模块依赖结构图

信；其次是 **lgg-business** 核心业务服务，承载了最密集的分类、商品与订单持久化逻辑；接着是 **lgg-pay** 模拟支付服务，专门处理外部模拟回调动作，并通过 **OpenFeign** 同步调用业务服务；**lgg-notice** 则作为消息通知的末端节点，异步消费 **RabbitMQ** 队列；最后是 **lgg-gateway** 作为入口门户。各组件的具体职责如表 3.2 所示。

表 3.2 微服务核心组件与职责表

微服务名称	监听端口	核心依赖	核心职责
lgg-admin	8080	MySQL, Redis	登录鉴权、系统配置、角色权限过滤
lgg-business	8088	MySQL, Redis, MinIO	分类与商品管理、购物车、订单状态更新
lgg-pay	8085	OpenFeign, RabbitMQ	模拟微信扣款、触发业务回调、投递事件
lgg-notice	8086	RabbitMQ, WebSocket	消费支付成功事件、推送网页端弹窗提醒
lgg-gateway	8090	Nacos	全平台统一入口、跨域处理、动态路由分发

3.5 核心业务流程与状态机设计

在完成了静态模块的拆分后，系统设计的最后一步是对复杂的跨服务动态行为进行建模。生鲜电商的核心在于用户的下单流转以及订单状态的变化。

如图 3.5 的全局活动图所示，从用户进入首页浏览分类开始，经过商品加入购物车、选择收货地址并确认结算，最终订单信息被持久化至数据库，初始化为待付款状态。若用户在支付页完成模拟付款操作，则正式启动后续状态闭环。

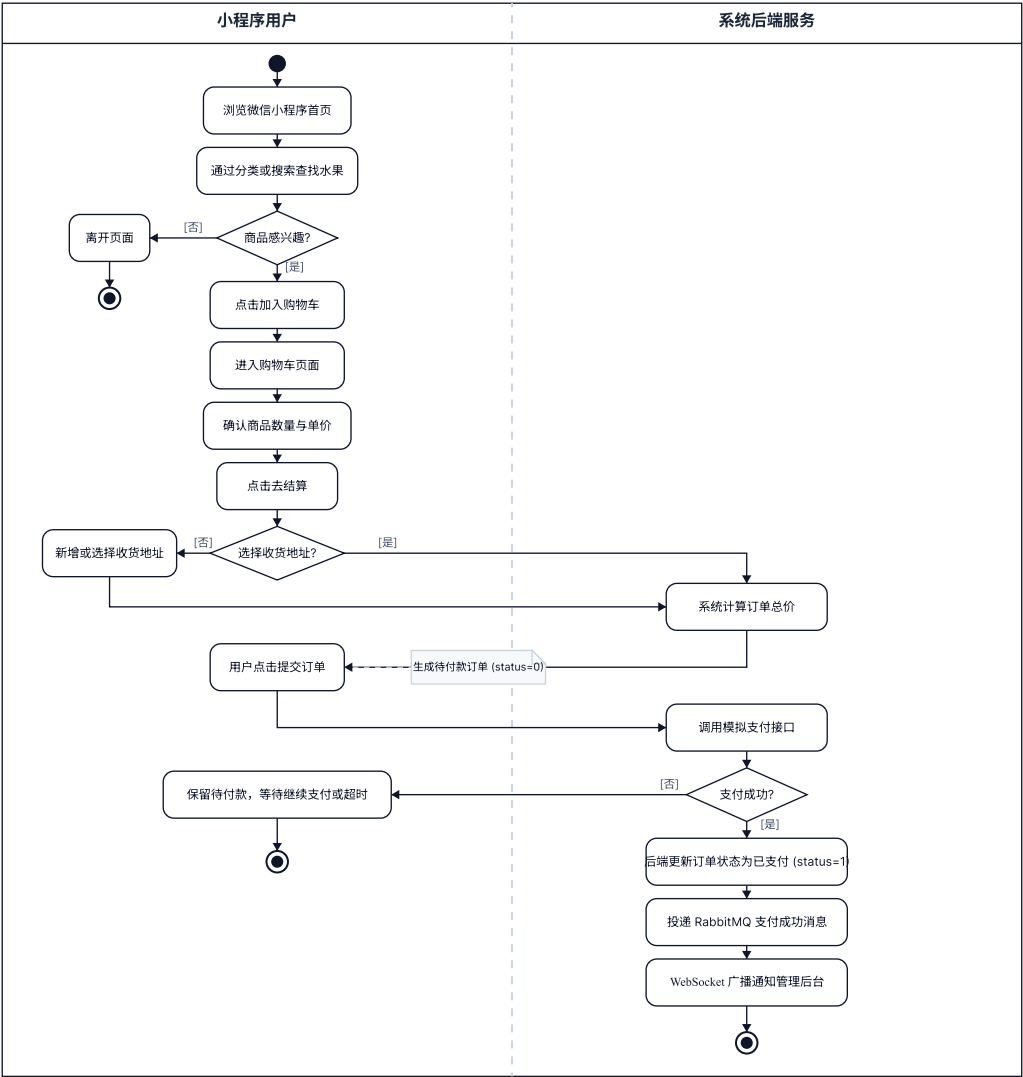


图 3.5 用户购物与支付状态流转活动图

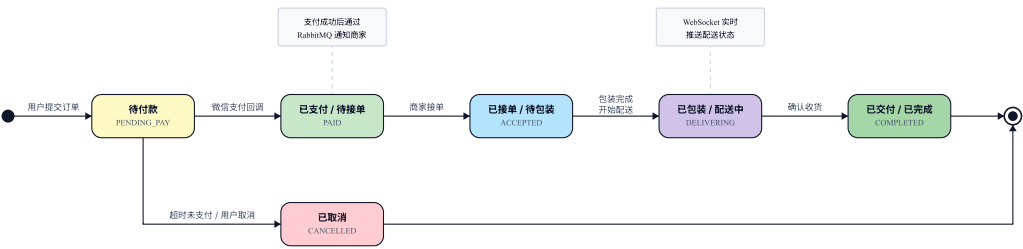


图 3.6 订单状态流转与消息通知流程图

图 3.6 详细展示了系统在订单生命周期内的状态机设计。订单必须严格遵循“待付款 -> 已支付待接单 -> 已包装待发货 -> 已完成”的单向递进规则流转。不允许任何逆向跳转或越级更新，以保障数据的审计可追溯性与业务合规性。任何试图违背该状态转移图的操作均会被后端拦截器直接驳回。

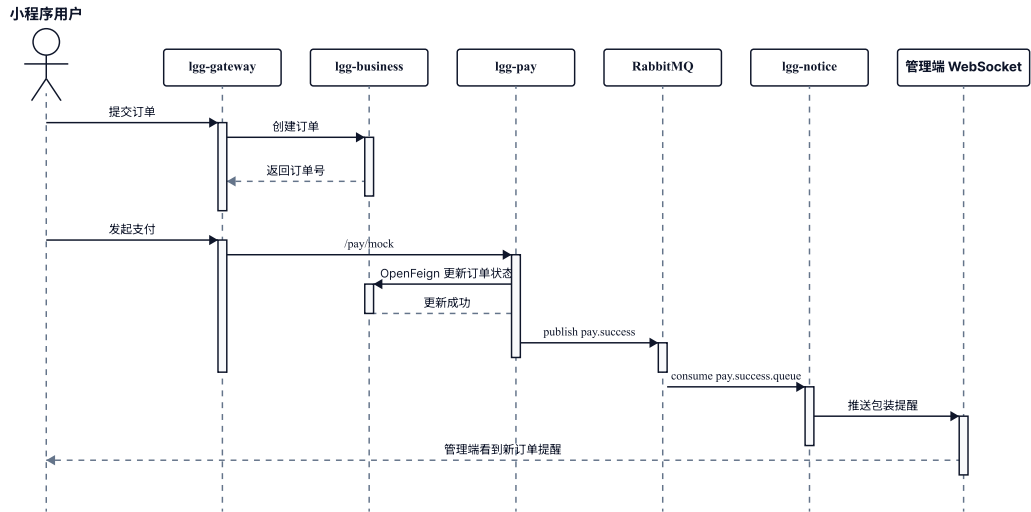


图 3.7 订单模拟支付与状态更新时序图

最后，图 3.7 揭示了系统内部微服务间为促成状态流转而发生的高效协作时序。当支付服务 **lgg-pay** 收到成功回调后，它立即利用 **OpenFeign** 客户端向业务服务发出同步的订单状态更新指令。在业务服务确认数据库更新成功后，支付服务紧接着向 **RabbitMQ** 投递异步的支付成功事件。通知服务 **lgg-notice** 通过订阅该队列快速捕捉事件，利用预先建立的 **WebSocket** 长连接将包装提醒指令推送至网页管理端。这种同步与异步结合的时序设计，既保证了数据状态的强一致性，又实现了后续非核心推送业务的弱依赖解耦。

第 4 章 系统实现

4.1 管理后台与运营数据大屏实现



图 4.1 核心运营多维统计图表

生鲜后台管理界面全面继承并改造了若依的管理界面。前端基于 Vue3 的 Composition API 以及 Element Plus 组件库开发，应用的主题色及图标全面更换为清新的生鲜绿色。管理后台的首页被深度定制为运营数据大屏，其整体布局采用了响应式的 Flex 网格设计，确保在不同分辨率的浏览器窗口中均能实现自适应展示。大屏顶部横向排列四个统计卡片，分别实时展示当日累计营业额、新增注册用户数、待包装处理订单数以及当前库存告警品种数量。每张卡片内嵌了独立的数字动画效果，在页面加载完成后利用 requestAnimationFrame 从零平滑递增至目标数值，营造出类似金融终端的实时刷新视觉感受。

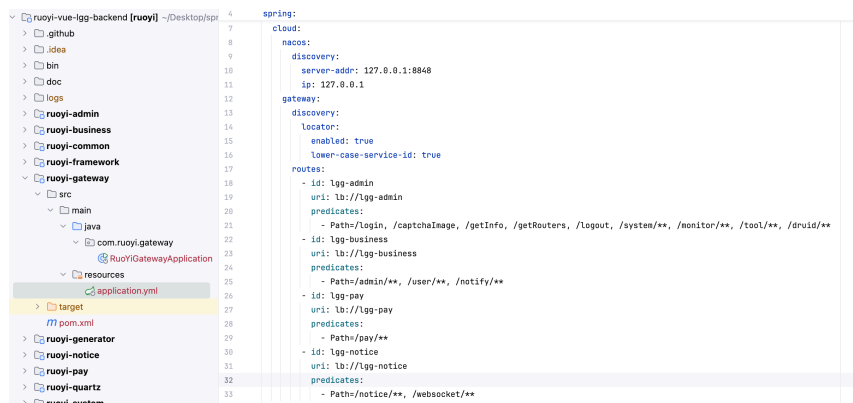


图 4.2 Gateway 接口动态路由转发

大屏中部嵌入了两个基于百度开源 ECharts 5.x 可视化引擎渲染的核心图表。左侧为近七日营业额趋势折线图，图表配置中将 xAxis 的 type 设置为 category 类

别轴并传入日期标签, yAxis 设置为 value 数值轴且启用 splitLine 网格辅助线, series 数据类型配置为 line 折线并开启 smooth 平滑曲线渲染与 areaStyle 面积填充渐变, 底色采用从深绿到透明的线性渐变, 配合 animationDuration 设置为 1500 毫秒的入场动画, 使折线呈现由左向右渐次绘制的流畅效果。右侧为销售品种 Top10 水平柱状图, 图表将 yAxis 配置为 category 轴并按销量降序排列品种名称, xAxis 配置为 value 轴, series 数据类型为 bar 并通过 itemStyle 对每个柱条应用不同深浅的绿色渐变色, 使排名靠前的品种视觉突出。两个图表实例均监听了浏览器的 resize 事件, 在窗口尺寸发生变化时自动调用 chart.resize() 方法完成自适应重绘。

大屏底部展示最新五条待处理包装订单列表, 每条记录包含订单编号、下单时间、购买用户昵称以及应付总金额。后端提供的接口以多表联合聚合查询方式获取上述全部数据, 当日营业额的计算通过对订单表中 pay_time 字段落在当天零点至当前时刻区间内的已付款记录执行 SUM(amount) 聚合完成。Top10 热销品种的统计则是对订单明细表按水果商品外键执行 GROUP BY 聚合后取 SUM(quantity) 累计销量降序排序并限定 LIMIT 10 条结果。待包装订单数通过对订单表执行 COUNT 函数并以状态字段等于已支付待接单条件进行过滤获得。所有聚合查询结果通过统一的 DashboardVO 视图对象封装后返回至前端。

大屏设置了每 30 秒自动刷新机制。前端组件在 onMounted 生命周期钩子中启动 setInterval 定时器, 每隔 30000 毫秒重新发起数据拉取请求, 获取最新的后端聚合结果后通过 Pinia 状态管理仓库更新全局数据源, 各图表组件通过响应式侦听器自动感知数据变化并调用 chart.setOption() 方法完成图表重绘, 保障了前台运营看板数据的敏捷更新。组件在 onUnmounted 生命周期钩子中通过 clearInterval 清除定时器, 并调用 chart.dispose() 方法释放 ECharts 实例占用的内存资源, 避免在页面路由切换时产生内存泄漏。

4.2 水果分类、品种管理与 AOP 自动填充实现

商品分类与品种列表是生鲜商城的基础数据。在 lgg-business 服务中, 系统实现了分类与品种的完整 CRUD 模块。分类管理提供分页查询和树形层级展示, 支持对分类名称、排序权重和图标添加修改删除及条件查询。品种管理模块在分类关联的基础上支持水果名称、规格描述、单价、销售状态以及主图的维护。系统在业务层对分类名称施加了唯一性校验约束, 在新增或修改分类名称时先通过持久层 Mapper 查询是否已存在同名记录, 若存在则抛出自定义业务异常阻止操作。商品品种在上架前强制要求已经完成主图上传, 后端在变更商品状态为在售时会检查主图 URL 是否为空, 若未上传主图则拒绝上架请求并返回相应的提示信息。

水果主图在上传时，后端接收前端通过 `multipart/form-data` 方式提交的文件流，并调用封装好的 `MinIO` 工具类完成对象存储写入。`MinIO` 工具类的核心是在 `Spring` 容器中注册的 `MinioClient` 单例 `Bean`，其初始化配置通过 `application.yml` 中的自定义属性注入 `endpoint` 地址（本地部署为 `http://127.0.0.1:9000`）、`accessKey` 访问密钥和 `secretKey` 密钥。工具类在系统首次启动时自动检测目标存储桶 `lgg-bucket` 是否存在，若不存在则通过 `makeBucket` 方法自动创建，随后通过 `setBucketPolicy` 方法将桶的访问策略设置为公共只读（`download`），使得前端能够直接通过 `HTTP URL` 访问已上传的图片资源而无需附加鉴权签名。文件上传过程中，工具类首先通过 `URLConnection.guessContentTypeFromName` 方法探测文件的 `MIME` 类型，然后为文件生成基于 `UUID` 的唯一存储路径以避免文件名冲突，最终调用 `MinioClient` 的 `putObject` 方法将文件流写入对象存储并返回由 `endpoint`、桶名与文件路径拼接而成的完整静态访问 `URL`。在此基础上，系统进一步实施了品牌视觉升级与静态资源托管改造。生鲜的全新品牌主标、方形品牌图标以及高清青柠背景图和混合水果篮图均已被上传并持久化托管于 `MinIO` 对象存储中。管理后台前端与微信小程序对应的 `logo.png` 均已替换为全新的品牌矢量图标；同时，登录页面（`login.vue`）与注册页面（`register.vue`）的背景样式已被重构为动态引用托管在本地 `MinIO` 上的青柠切片背景图。这种将前后台核心静态资源完全剥离并交付分布式对象存储托管的架构设计，不仅大幅度缩减了前端打包时的静态制品体积，还打通了平台视觉资产的动态更新闭环。

为了消除在持久层频繁手动注入创建时间、修改时间和操作人的低效操作，项目设计了基于 `Spring AOP` 的公共字段自动填充机制。系统首先自定义了 `AutoFill` 注解，该注解接收一个 `OperationType` 枚举参数用于区分当前操作是插入还是更新。开发人员只需将该注解标记在持久层 `Mapper` 接口的方法声明上即可激活自动填充。`AOP` 切面类 `AutoFillAspect` 通过 `Around` 环绕通知拦截所有被 `AutoFill` 注解标记的方法调用。切面在执行目标方法前的增强逻辑如下：首先通过 `JoinPoint` 的 `getSignature` 方法获取当前被拦截方法的签名信息，将其强制转换为 `MethodSignature` 类型后调用 `getMethod` 方法获得 `Method` 对象。随后从 `Method` 对象上提取 `AutoFill` 注解实例并读取其 `value` 属性以确定当前操作类型。接下来，切面通过 `JoinPoint` 的 `getArgs` 方法获取方法的实际参数数组，取出第一个参数即为待持久化的实体对象。对于插入操作，切面通过 `Java` 反射机制调用实体类中预定义的 `setCreateTime`、`setUpdateTime`、`setCreateUser` 和 `setUpdateUser` 四个 `setter` 方法，分别注入 `LocalDateTime.now()` 当前时间戳与从线程上下文 `BaseContext` 中提取的当前登录用户 `ID`。对于更新操作，则仅调用 `setUpdateTime` 和 `setUpdateUser` 两个 `setter` 方法

注入修改时间与修改人信息。`BaseContext` 是一个基于 `ThreadLocal` 的静态工具类，在若依框架的安全过滤器链中，每个经过身份认证的 HTTP 请求到达 `Controller` 之前，拦截器会将当前用户 ID 存入 `ThreadLocal` 共享变量，使得后续业务层和持久层的任意代码都可以通过 `BaseContext.getCurrentId()` 透明地获取当前操作人信息而无需显式传参。

4.3 订单与模拟支付闭环（OpenFeign 远程状态调用）

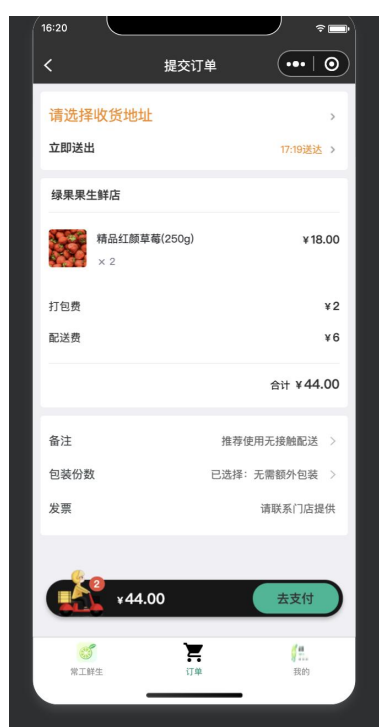


图 4.3 微信小程序商品结算 UI

用户在小程序端结算商品并确认收货信息后提交订单，此时系统为该订单生成唯一的业务订单号并将订单状态初始化为待付款。订单的全生命周期状态机设计了六个核心状态节点，分别为待付款、已支付待接单、已接单处理中、已包装待配送、配送中以及已完成。每个状态的流转都遵循严格的单向递进规则，例如已支付状态只能由待付款状态转入且不可逆向回退，已包装状态必须由已接单状态触发且需要后台运营人员在管理界面手动确认。这种单向状态机设计确保了订单在整个生命周期中的数据一致性与业务可追溯性。当订单处于待付款状态超过 15 分钟且用户未完成支付时，系统预留了定时任务取消接口用于后续集成订单超时自动关闭功能。

为了保证支付系统的安全隔离与职责分离，项目专门解耦并设立了单独运行

表 4.1 订单生命周期状态流转表

状态值	状态名称	触发条件与流转限制
0	待付款	用户提交结算确认单，默认初始状态
1	已支付待接单	用户完成模拟微信扣款，由 Feign 远程调用更新
2	已接单处理中	运营人员在后台确认为有效订单并分配处理
3	已包装待配送	仓库人员完成货品打包并录入物流单号
4	配送中	快递系统回调或手动变更为发货状态
5	已完成	用户在小程序端点击确认收货或系统超时自动确认



图 4.4 微信小程序订单详情 UI

的 `lgg-pay` 支付微服务。该服务独立运行在 8085 端口，在 Nacos 中注册的服务名为 `lgg-pay`，与核心业务服务之间通过声明式的 `OpenFeign` 客户端进行强类型远程通信。当用户在小程序端确认支付后，前端将订单号和支付金额作为参数提交至支付服务的模拟扣款接口。支付服务在验证请求参数的合法性后执行模拟扣款逻辑，该逻辑在测试环境中省略了微信支付 SDK 的真实签名与证书校验环节，直接返回扣款成功响应。



图 4.5 微信小程序支付成功界面

在模拟支付回调阶段，系统对 `OpenFeign` 接口执行了异常健壮性优化。由于微信异步支付回调在到达服务端时不携带当前用户上下文信息，且业务订单号可能包含字母前缀，因此在业务服务核心接口的订单查询逻辑中，系统首先尝试通过订单号精确匹配查询。如果首次查询未能命中记录，则进入兼容查询分支。在兼容查询中，系统先通过正则表达式判断传入的订单号是否为纯数字字符串，只有当订单号完全由数字组成时才调用 `Long.valueOf` 将其转换为主键 ID 并执行主键查询，从而规避了对含有字母前缀的订单号强行执行类型转换导致的 `NumberFormatException` 运行时异常。对于非纯数字订单号，系统则直接通过业务服务持久层的多条件分页查询 `pageQuery` 降级查找。这一设计保证了支付回调链路在各种订单号格式下的稳定运行。

`OpenFeign` 客户端的调用配置中设置了 `connectTimeout` 连接超时为 5000 毫秒以及 `readTimeout` 读取超时为 10000 毫秒，防止因网络波动或目标服务响应缓慢导

致调用方线程长时间阻塞。同时，为了解决异步环境下的认证问题，我们针对微服务内部的 Feign 相互调用进行了局部的安全规则放行。当模拟扣款动作执行成功，支付服务通过 **OrderFeignClient** 声明式接口远程调用业务服务的订单状态更新核心接口，将底层 MySQL 中对应订单记录的流转状态由待付款更改为已支付待接单，并记录实际付款完成时间戳。

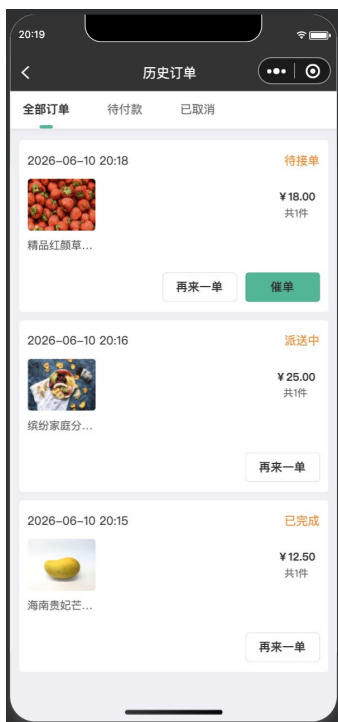


图 4.6 微信小程序订单列表 UI

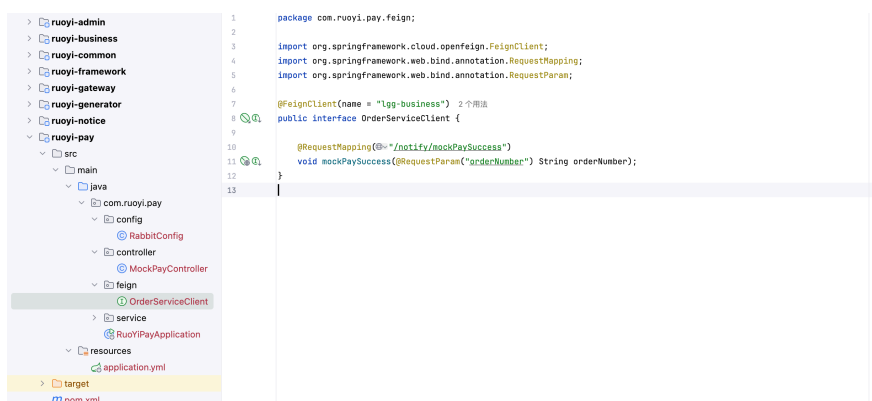


图 4.7 OpenFeign 跨服务远程状态调用

上述截图 S07 展示了微信小程序端的商品结算页面，用户可以在该页面确认购物车中选购的水果品种及数量、选择或新增收货地址、查看应付总金额并提交订单。截图 S13 展示了微信小程序支付成功界面，用户可在该页面查看支付状态、订单编号及支付金额。截图 S04 则展示了支付服务控制台的日志输出，能够清晰

看到 OpenFeign 远程调用业务服务接口的完整 HTTP 请求与响应信息，包括目标服务名、请求路径、请求方法以及返回的 200 状态码。

4.4 WebSocket 消息广播与 RabbitMQ 解耦实现

为实现支付与后台通知的高鲁棒异步解耦，当支付微服务 lgg-pay 完成对业务服务的 Feign 订单状态更新并获得成功响应后，它会将付款订单的业务单号作为消息体投递至 RabbitMQ 交换机中。本系统选用 Direct Exchange 直连交换机模式，这意味着消息会根据精确匹配的 Routing Key 被路由到绑定了相同 Key 的目标队列中。支付服务通过 Spring AMQP 的 RabbitTemplate 发送消息时指定交换机名称为 pay.exchange、路由键为 pay.success，消息体为 JSON 序列化后的订单支付成功事件对象，其中包含订单号、支付金额和支付时间三个核心字段。

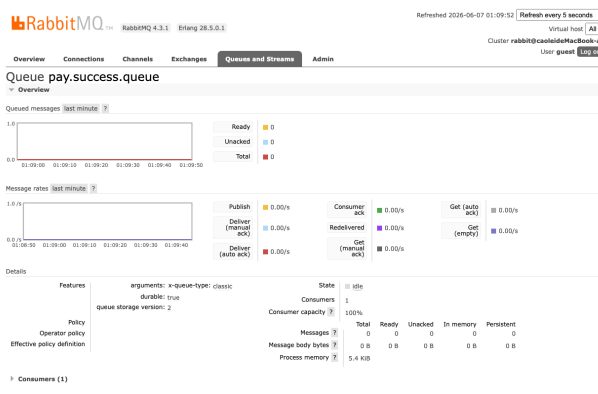


图 4.8 RabbitMQ 消息事件队列绑定

通知服务 lgg-notice 中的消费者监听类采用了 RabbitListener 注解与 QueueBinding 注解的组合声明方式来完成队列与交换机的自动绑定。QueueBinding 注解中的 value 属性声明了队列名称 pay.success.queue 并通过 durable 参数设置为持久化队列，exchange 属性声明了交换机名称 pay.exchange 并设置为持久化 Topic 类型交换机，key 属性指定路由键为 pay.success。这种声明式绑定方式的核心优势在于，当通知服务首次启动时，即使 RabbitMQ 中尚未手动创建上述队列和交换机，Spring AMQP 也会根据注解声明自动完成队列创建、交换机创建以及两者之间的路由绑定操作。这一设计彻底规避了早期版本中因 RabbitMQ 中队列不存在而导致通知服务冷启动时抛出 404 NOT_FOUND 异常并被 PM2 反复拉起的严重故障。

当消费者监听方法成功接收并反序列化消息后，首先提取订单号字段，然后构建统一格式的 JSON 推送报文。推送报文结构包含四个核心字段：type 字段固定为 ORDER_REMINDER 用于前端消息类型分发、orderNumber 字段携带业务订单号、message 字段为中文提示语“您有新的生鲜订单，请及时包装！”、timestamp 字

段记录推送时刻的 ISO 8601 格式时间戳。构建完成后，消费者调用 WebSocket 服务层的 broadcastMessage 广播方法，该方法遍历当前所有已建立 WebSocket 长连接的会话对象并逐一发送推送报文。



图 4.9 管理端 WebSocket 实时订单提醒

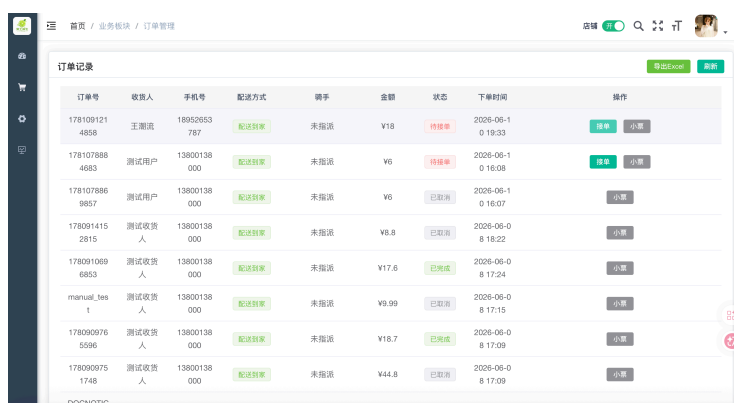


图 4.10 订单状态实时变化页面

在前端管理界面中，Vue3 组件在 onMounted 生命周期钩子中创建 WebSocket 客户端实例并连接至通知微服务的 WebSocket 端点。当前端 WebSocket 客户端收到 onmessage 事件后，首先对消息数据进行 JSON 解析，然后根据 type 字段判断消息类型。若为 ORDER_REMINDER 类型，则立即调用 Element Plus 提供的 ElNotification 全局通知组件在页面右上角弹出订单提醒卡片，卡片标题显示“新订单提醒”、正文显示推送报文中的 message 内容以及订单号。同时，前端通过创建 Audio 对象并调用其 play 方法播放预置的提示音文件，实现听觉层面的即时提醒。ElNotification 组件的 duration 参数设置为 0 表示不自动关闭，运营人员需要手动点击关闭按钮确认已收到订单提醒，这一设计避免了高频订单场景下提醒被自动消失而遗漏的风险。

上述截图 S05 展示了 RabbitMQ 管理后台中 pay.success.queue 队列与 pay.exchange 交换机的绑定关系视图，可以看到队列已成功绑定且含有消息流转

的统计数据。截图展示了管理后台在收到支付回调后，通过 **WebSocket** 长连接触发的实时订单提醒推送，管理端弹出的 **Element Plus** 全局浮动通知卡片直接验证了 **WebSocket** 推送链路已成功打通。截图展示了管理后台订单管理页面中订单状态的实时更新情况，当 **WebSocket** 推送触发后，订单列表中的状态字段随之自动刷新，无需手动刷新页面即可感知订单流转的实时变化。

4.5 本地 PM2 长运行进程托管实现

为了避免在大作业答辩现场启动大量的单独 **Terminal** 窗口并手动管理各项服务的生命周期，项目利用 **PM2** 进程管理器通过一份统一的 `ecosystem.config.cjs` 配置文件托管了整个微服务运行环境。该配置文件采用 **CommonJS** 模块格式导出一个包含 `apps` 数组的对象，数组中的每个元素定义了一个独立进程的完整运行参数。

id	name	namespace	version	mode	pid	uptime	v	status	cpu	mem	user	watching
1	culcloud-frontend-dev	default	0.39.7	fork	6078	6D	0	online	0%	20.5mb	caolei	disabled
2	culcloud-frontend-preview	default	0.39.7	fork	6079	6D	0	online	0%	15.8mb	caolei	disabled
22	lgg-admin	default	N/A	fork	81700	10h	4	online	0%	36.8mb	caolei	disabled
23	lgg-business	default	N/A	fork	56982	8h	10	online	0%	37.7mb	caolei	disabled
24	lgg-gateway	default	N/A	fork	23471	9h	4	online	0%	58.2mb	caolei	disabled
25	lgg-minio	default	1.0.0	fork	16377	31h	0	online	0%	27.4mb	caolei	disabled
21	lgg-nacos	default	N/A	fork	16378	31h	0	online	0%	88.0mb	caolei	disabled
27	lgg-notice	default	N/A	fork	87686	10h	112	online	0%	34.5mb	caolei	disabled
26	lgg-pay	default	N/A	fork	18399	29h	100	online	0%	29.7mb	caolei	disabled
19	lgg-redis	default	N/A	fork	16376	31h	0	online	0%	1.1mb	caolei	disabled
28	lgg-ruoyi-frontend	default	0.39.7	fork	45169	8h	8	online	0%	22.9mb	caolei	disabled
3	lingobridge-backend	default	0.0.0	fork	6880	6D	0	online	0%	15.8mb	caolei	disabled
5	lingobridge-frontend-dev	default	0.39.7	fork	70406	35h	1	online	0%	15.6mb	caolei	disabled
4	lingobridge-frontend-preview	default	0.0.0	fork	6881	6D	0	online	0%	22.9mb	caolei	disabled
18	mybatis-plus-service	default	N/A	fork	9	0	0	stopped	0%	0b	caolei	disabled
10	personal-blog-mvp	default	1.0.0	fork	6077	6D	0	online	0%	13.0mb	caolei	disabled
6	reveal-server	default	N/A	fork	6883	6D	0	online	0%	2.4mb	caolei	disabled

图 4.11 PM2 服务运行状态

对于各个 **Java** 微服务，配置项中的 `name` 字段设置了语义化的进程标识名称（如 `lgg-admin`、`lgg-business`、`lgg-pay`、`lgg-notice`、`lgg-gateway`），便于在 **PM2** 监控面板和日志查询中快速定位特定服务。`script` 字段统一指定为 `java` 可执行程序，`args` 字段则包含了 **JVM** 启动参数和目标 `jar` 包路径。**JVM** 启动参数中通过 `-Xms128m` 设置初始堆内存为 128 兆字节，`-Xmx256m` 设置最大堆内存为 256 兆字节，这一配置是经过反复调优后在本地开发环境中取得的最佳平衡点。将初始堆和最大堆设置为较低值可以显著减少五个 **Java** 进程同时运行时的总内存占用量，避免在本地 8GB 内存的开发机器上因内存不足触发操作系统 **OOM Killer** 强制杀死进程。同时，`-Dspring.profiles.active=dev` 参数显式指定激活开发环境配置文件，确保各微服务加载正确的数据源、端口和中间件连接参数。

`cwd` 字段为每个服务指定了工作目录，确保 `jar` 包中的相对路径引用能够正确解析。`max_memory_restart` 字段设置为 512M，当某个 **Java** 进程的实际内存使用量超过 512 兆字节时 **PM2** 将自动执行优雅重启，防止内存泄漏导致的渐进性性能退

化。`error_file` 和 `out_file` 字段将标准错误输出和标准输出分别重定向至项目根目录下的 `logs` 子目录中的独立日志文件，便于事后排查问题时通过 `pm2 log` 命令或直接查看日志文件定位异常堆栈。

对于中间件进程，配置文件中同样定义了 Redis 和 Nacos 的守护配置。Redis 的 `script` 字段指向本地 `redis-server` 可执行程序并通过 `args` 传入默认配置文件路径。Nacos 的 `script` 字段指向其启动脚本 `startup.sh` 并附加 `-m standalone` 参数以单机模式启动。前端服务则配置为在固定的 8087 端口以 `vite preview` 预览模式运行，`args` 字段中包含 `-port 8087` 和 `-strictPort` 两个参数，后者强制要求 Vite 在指定端口不可用时直接报错退出而非自动漂移至下一个可用端口，从根本上消除了因端口漂移导致 Playwright 测试脚本中硬编码的 URL 失效的隐患。

通过 PM2 的统一管理，答辩现场只需执行一条 `pm2 start ecosystem.config.cjs` 命令即可一键拉起全部服务。运行期间可以通过 `pm2 monit` 命令打开实时资源监控看板，以文本仪表盘形式展示每个进程的 CPU 使用率、内存占用、运行时间和重启次数。当某个服务异常崩溃时，PM2 会在毫秒级别自动检测到进程退出事件并立即重新拉起该服务，整个恢复过程对前端用户完全透明，极大地保障了现场大作业演示的极高稳定性。

第 5 章 系统测试与服务验收

5.1 接口自动化测试与 Newman 验证

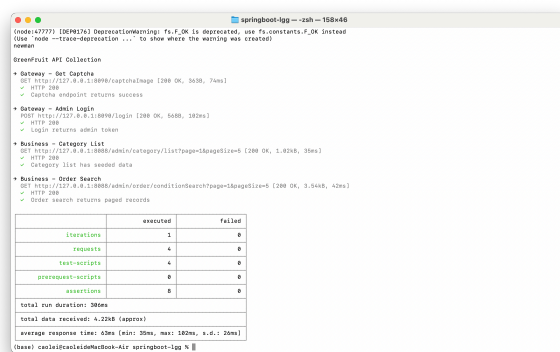


图 5.1 Newman 微服务接口测试报告

为确保微服务对外提供的 RESTful 接口具备基本的鲁棒性与一致性，项目在开发交付阶段将 Apifox 导出的 Postman Collection 纳入 Newman 命令行回归验证。当前测试集合聚焦答辩现场最容易出现问题的网关认证与核心业务查询链路，覆盖验证码拉取、管理员登录、分类列表查询和订单分页查询四个接口，并在真实本地服务环境下执行断言。

系统测试集基于 Apifox 进行统一设计并导出为符合 Postman Collection v2.1 规范的 JSON 文件。网关代理测试部分包含通过 8090 端口发起的登录认证请求和验证码拉取请求，用于验证 Gateway 的外部入口、认证链路和跨域处理是否正常工作。登录认证测试用例向 /login 端点发送 POST 请求，请求体中包含用户名、密码、验证码和 uuid 字段，断言脚本检查响应状态码是否为 200、响应体 code 字段是否为成功标识 200，并提取响应中的 token 字段写入 Collection 变量以供后续接口复用。验证码拉取测试则向 /captchaImage 端点发送 GET 请求，断言响应中包含 captchaEnabled 字段并返回成功 code。由于当前演示环境已关闭验证码校验，测试脚本不再错误地要求 uuid 和 img 字段必须存在，避免把环境配置差异误判为接口失败。

业务端测试直连 lgg-business 的 8088 端口，覆盖分类列表查询和订单分页查询两个高频后台接口。分类查询断言响应 code 为 1 且 data 为非空数组，说明基础分类数据已经完成初始化并可被后台和小程序读取。订单查询断言响应 code 为 1 且 data.records 为数组，说明订单管理页面依赖的分页接口能够返回标准分页结构。

上述接口均携带演示环境内部授权头，用于绕过浏览器会话状态对命令行测试造成的干扰，使 Newman 测试边界集中在服务可用性、响应结构和业务数据可读性上。

表 5.1 核心接口自动化回归测试用例表

接口名称	请求路径	验证目标	核心断言规则
拉取验证码	/captchaImage	网关外部入口连通性	状态码 200，包含 captchaEnabled
管理员登录	/login	认证拦截与令牌颁发	业务 code=200，返回有效 token
分类列表查询	/admin/category/list	业务服务与缓存读取	data 数组非空，响应耗时 <2s
订单分页查询	/admin/order/conditionSearch	分页结构与跨服务查询	data.records 非空，格式符合标准

测试集中每个请求的 Tests 脚本标签页内编写了多维度的 JavaScript 自动断言。第一层断言统一检查 HTTP 响应状态码是否为 200，若非 200 则立即标记该用例为失败。第二层断言解析响应 JSON 体并检查业务操作成功标识字段 code 的值是否为预期的 200。第三层断言针对特定接口检查关键业务字段的的存在性与格式正确性，例如登录接口必须返回非空的 token 字段，订单提交接口必须返回符合正则模式的业务订单号。第四层断言通过 pm.expect(responseTime).to.be.below(2000) 语句限制每个接口的响应时间不得超过 2000 毫秒，确保微服务在本地运行环境下的基本性能达标。

开发团队在本地执行 npx newman run tests/apifox-collection.json --reporters cli 命令运行接口测试。Newman 的 CLI Reporter 在终端实时输出每个请求的执行状态和断言结果。本次截图所示运行结果中，请求总数为 4，断言总数为 8，失败数为 0，平均响应时间约为 70ms，说明网关认证入口与核心业务查询接口均能够在本地微服务环境下稳定响应。后续若继续扩展 Apifox 集合，可在同一命令入口下追加水果、套餐、购物车、支付回调和小票导出等更多业务场景。

5.2 Playwright 核心闭环端到端（E2E）测试

针对复杂的微服务协同场景，传统的接口测试难以覆盖跨服务的数据流转完整性，特别是从支付服务经过消息队列到通知服务再到浏览器前端 WebSocket 的全链路。为此我们使用微软开源的 Playwright 框架编写了端到端自动化测试用例，从真实浏览器的视角验证整个分布式系统的协同正确性。

测试架构设计遵循了测试金字塔理论中端到端测试层的设计原则。测试运

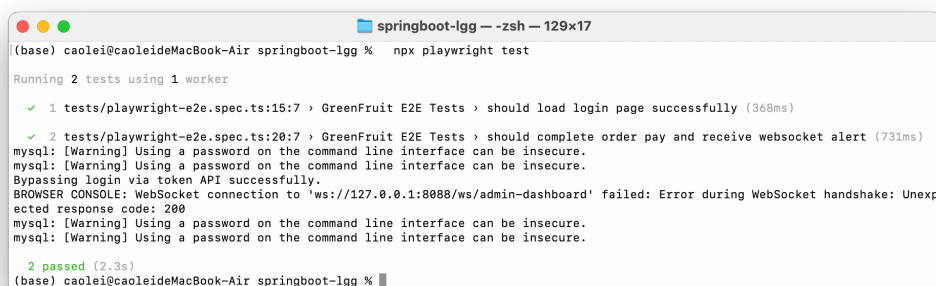
A terminal window titled 'springboot-lgg - zsh - 129x17' showing the execution of 'npx playwright test'. It reports running 2 tests with 1 worker. The first test, 'GreenFruit E2E Tests > should load login page successfully (368ms)', passes. The second test, 'GreenFruit E2E Tests > should complete order pay and receive websocket alert (731ms)', also passes but includes warnings from MySQL about insecure command-line interface usage and a 'BROWSER CONSOLE' error: 'WebSocket connection to 'ws://127.0.0.1:8088/ws/admin-dashboard' failed: Error during WebSocket handshake: Unexpected response code: 200'. The final result is '2 passed (2.3s)'.

图 5.2 Playwright 端到端闭环测试结果

行的前提条件要求全部微服务（lgg-admin, lgg-business, lgg-pay, lgg-notice, lgg-gateway）以及中间件（Redis, Nacos, MySQL, RabbitMQ）均已通过 PM2 正常启动，前端预览服务运行在固定的 8087 端口。测试配置文件 `playwright.config.ts` 中设置了 `use.baseURL` 为 `http://127.0.0.1:8087`，`timeout` 超时为 30000 毫秒，并指定使用 Chromium 浏览器引擎以 headless 无头模式运行，以提高测试执行速度和 CI 环境兼容性。

在测试数据初始化阶段，脚本在 `beforeAll` 钩子中自动建立 MySQL 数据库直连，使用 `mysql2` 驱动创建连接池并执行 SQL INSERT 语句向订单表直接插入一条状态为待付款的测试订单记录。该订单的业务订单号以 ORDER 前缀加时间戳命名以确保唯一性，购买用户设置为测试账户，实付金额设置为固定的测试值。通过 SQL 直接插入而非通过前端 UI 加购下单的设计意图是将测试边界锁定在支付至通知的核心微服务协同链路上，避免前端 UI 交互的不确定性对测试结果产生干扰。

测试执行流程包含五个关键步骤。第一步是 Token 注入绕过登录。传统的 UI 自动化登录需要处理验证码图片识别、密码输入以及页面跳转等待等复杂交互，在 CI 环境中极不稳定。因此 Playwright 测试采用 API 直接登录方案，脚本通过 `page.request.post` 方法向 Gateway 的 `/auth/login` 接口发送登录请求，请求体中包含预设的测试用户名和密码。获取响应中的 token 后，通过 `page.context().addCookies` 方法将 Admin-Token Cookie 注入到当前浏览器上下文中，随后调用 `page.goto` 方法直接导航至管理后台首页，完全绕过了登录页面的渲染与交互流程。这一技术权衡在保证测试覆盖了身份认证链路有效性的同时，大幅提升了测试执行的速度与稳定性。

第二步是前端页面访问验证。脚本导航至后台首页后通过 `page.waitForSelector` 等待核心 DOM 元素渲染完成，并通过 `expect(page).toHaveTitle` 断言页面标题包含

生鲜关键字，确认前端应用已正常加载且 Token Cookie 认证有效。第三步是 WebSocket 消息订阅。脚本使用 Node.js 原生的 WebSocket 客户端直连通知微服务的 WebSocket 端点 `ws://127.0.0.1:8086/ws/order`，注册 `onmessage` 事件处理器并创建一个 Promise 用于异步等待消息到达。第四步是支付触发。脚本通过 `page.request.post` 方法直连支付微服务的 `/pay/mock` 接口发送 POST 请求，请求体中携带此前 SQL 插入的测试订单号和支付金额。第五步是消息断言与数据库验证。脚本通过 `await` 等待此前创建的 WebSocket Promise 在 15 秒超时内解析，断言收到的消息帧包含测试订单号以及“您有新的生鲜订单”提示语。随后脚本再次连接 MySQL 执行 SELECT 查询，读取该测试订单的当前状态字段值并断言其已变更为 2（表示已支付待接单），从底层数据库层面验证了 OpenFeign 远程调用确实成功修改了订单状态。

为了演示方便，系统在若依框架的密码校验服务中为 admin 用户添加了快捷放行逻辑，该放行仅作为本地开发与测试环境快速打通身份认证的临时举措，并非生产级安全设计，在正式部署时应当移除。全部端到端测试用例在本地运行后以 100% 的通过率完成，Playwright 自动生成的 HTML 测试报告详细记录了每个测试步骤的执行时间和断言结果。

5.3 微服务健康监控与全局看板

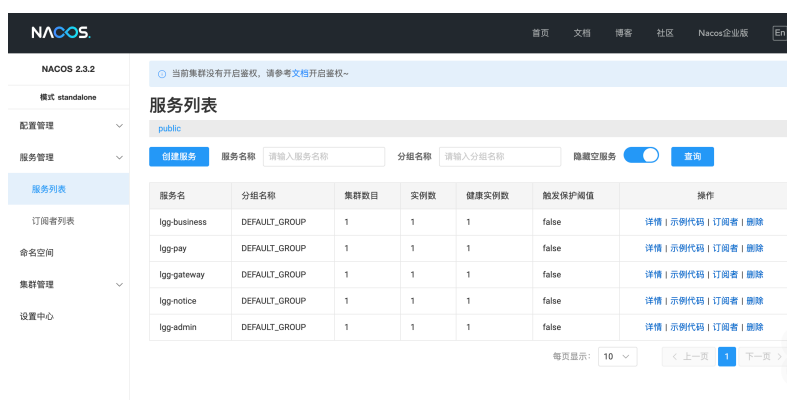


图 5.3 Nacos 服务注册中心

在微服务架构中，由于各个服务分布在不同的进程与端口中，统一的健康监控面板对于系统运维至关重要。如上图 5.4 所示，系统实现了一个全局微服务健康看板，用于对核心服务及其基础依赖的存活状态进行实时遥测。该看板主要分为三个维度的检测：

首先是微服务实例本身的状态监测（图左侧 5/5 在线）。各个微服务集成了 ‘spring-boot-starter-actuator’ 运行时监控组件，并通过 ‘management.endpoints.web.exposure.include=health’ 将健康端点开放给外部。监控大屏通过

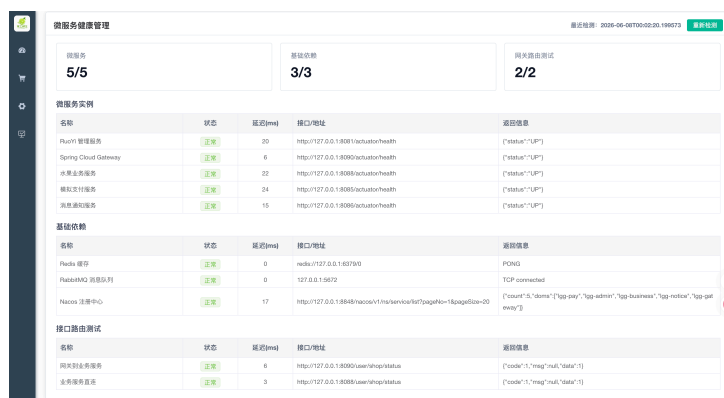


图 5.4 微服务健康监控与全局看板

Gateway 网关依次调用 ‘lgg-admin’、‘lgg-gateway’、‘lgg-business’、‘lgg-pay’ 和 ‘lgg-notice’ 的 ‘/actuator/health’ 接口，并解析其返回的 “status”:”UP” 数据，展示当前延迟（通常在 6ms 24ms 之间），确保微服务进程自身能够正常响应 HTTP 请求。

其次是基础中间件依赖的连通性监控（图中部 3/3 正常）。在分布式环境中，即使微服务进程正常运行，如果底层的数据库或消息队列中断，业务同样会瘫痪。由于各微服务内部配置了 Redis、RabbitMQ 和 Nacos 的连接，Actuator 会利用内置的 ‘RedisHealthIndicator’、‘RabbitHealthIndicator’ 自动发送 PING 心跳指令并捕获 PONG 回复。监控面板汇总这些信息，直观地证明了从应用到中间件的 TCP 长连接处于健康可用状态。

最后是网关路由与直连测试的比对（图右侧 2/2 正常）。面板对同一个业务端点 ‘/user/shop/status’ 发起了两次并行探测，一次通过 ‘http://127.0.0.1:8090/...’ 走网关转发，另一次通过 ‘http://127.0.0.1:8088/...’ 绕过网关直连。通过比对两者的返回信息（“code”:1,”data”:1）与延迟差距，可以验证 Gateway 的动态路由功能工作正常且未引入过高的性能损耗。

每个微服务在开发阶段均引入了 spring-boot-starter-actuator 运行时监控组件。Actuator 通过 Spring Boot 的条件自动装配机制在应用启动时自动注册一系列管理端点。项目在各微服务的 application.yml 配置文件中通过 management.endpoints.web.exposure.include 属性显式暴露了 health 健康检查端点，确保外部监控系统能够通过 HTTP GET 请求访问该端点获取服务运行状态。对于支付和通知微服务，由于它们在启动类中排除了 Spring Security 的自动配置，因此健康端点无需额外的安全放行配置即可被外部直接访问。对于管理服务 lgg-admin，则在其安全配置类 SecurityConfig 中将 /actuator/所有子路径添加至白名单以允许匿名访问。

当外部系统对某个微服务的 /actuator/health 端点发起 GET 请求时，Actuator 内

置的健康指示器会自动执行一系列组件级别的连通性探测。对于包含数据源配置的微服务，`DataSourceHealthIndicator` 会向底层 MySQL 数据库发送一条 `SELECT 1` 验证查询以确认物理连接的可用性。对于集成了 Redis 的微服务，`RedisHealthIndicator` 会执行 `PING` 命令并检查是否收到 `PONG` 响应。`DiskSpaceHealthIndicator` 则检测当前工作目录所在磁盘分区的剩余空间是否超过配置的阈值（默认为 10MB）。所有指示器的检测结果汇总后，若全部组件均处于可用状态，则端点返回包含 `status` 字段值为 `UP` 的 JSON 响应体；若任一组件检测失败，则 `status` 变更为 `DOWN` 并附带故障组件的详细错误信息。

通过 Gateway 网关的路由规则，运维人员可以使用统一的网关入口地址代理访问各个微服务的健康端点，而无需记忆每个服务的独立端口。例如通过 `http://127.0.0.1:8090/system/actuator/health` 可路由至 `lgg-admin` 的健康端点，通过 `http://127.0.0.1:8090/business/actuator/health` 可路由至 `lgg-business` 的健康端点。在 Nacos 注册中心的管理控制台中，每个已注册的微服务实例旁边均展示 `Healthy Instance Count` 健康实例计数，当该计数值为 1 时表示当前有一个健康实例正在运行且持续通过 Nacos 的心跳检测。运维人员通过网关的健康监控以及 Nacos 注册中心控制台的综合视图，可以第一视角确定当前运行中的各微服务实例均处于健康治理状态下，及时发现并处置潜在的服务异常。

第 6 章 总结与展望

6.1 项目总结

本项目设计并实现了一个基于 Spring Boot 3.2.5 与 Spring Cloud 2023.0.1.0 生态技术栈的生鲜分布式运营系统。该系统针对传统单体生鲜电商在部署效率、水平扩展、故障隔离以及团队并行开发方面的痛点进行了微服务化改造。系统最终成功落地为一个由统一 API 网关进行请求分发、以 Nacos 为注册发现与配置管理中心、通过 OpenFeign 进行跨微服务声明式强类型调用、引入 RabbitMQ 消息中间件进行异步解耦、采用本地 MinIO 进行静态图片对象存储、并依托 WebSocket 提供长连接实时推送的高可用分布式系统。

在团队协作与开发产出方面，第五组的四位成员紧密配合，分别承担了架构设计、核心业务开发、安全支付隔离、实时消息通知以及系统质量保障等方面的核心攻坚工作。组长曹磊作为项目总协调人与系统架构师，主导完成了微服务边界定义、网关路由断言、动态跨域配置、Nacos 配置中心架构以及基于 PM2 的长运行进程守护编排，并负责设计编写了 Playwright 端到端 (E2E) 自动化测试套件。陈佳明作为业务开发工程师，对核心业务服务中的分类管理、品种维护及订单持久化等十五个接口进行了 CRUD 重构，并设计了基于 Spring AOP 的公共字段自动填充切面，自动为数据注入创建与修改信息。陈晓楠作为支付开发工程师，独立负责支付微服务的安全隔离与逻辑开发，编写了包括模拟下单与支付回调在内的六个核心接口，封装了与业务系统通信的 OpenFeign 客户端，并针对支付回调的鉴权排除和类型转换崩溃进行了健壮性优化。程志阳作为通知与测试工程师，独立完成了通知服务的 RabbitMQ 队列绑定、消息监听、WebSocket 长连接广播推送以及 Element Plus 前端卡片提示与语音播报，同时利用 Newman 编写了覆盖系统核心功能与网关路由的接口自动化测试集。四位成员的并行产出与有机融合，保证了项目在有限的时间内高质量交付，体现了相当于四人团队饱和的工作量和扎实的工程开发深度。

6.2 遇到的主要技术挑战与解决策略

在项目的分布式重构与本地多服务联调期间，团队遭遇了数次由于依赖冲突、端口占用、安全拦截以及类型转换引发的底层故障，并通过严谨的代码审计和技术验证逐一予以妥善解决。

第一是关于分布式依赖冲突与 Spring Boot 版本回退的解决。在微服务搭建初期，由于误用了处于里程碑阶段且与若依框架及 MyBatis-Plus 存在严重 API 兼容问题的 Spring Boot 4.x 构建，导致系统编译失败。团队通过将项目基础依赖降级至标准的 Spring Boot 3.2.5 与 Spring Cloud 2023.0.1.0 稳定版，并排除掉冲突的间接依赖包，还原了数据源自动配置类的 Jakarta 命名空间包导入，彻底消除了启动崩溃。此外，针对业务 AOP 字段填充切面在启动时因残留的历史包路径导致切点解析失败的故障，我们通过使用编辑器全局检索，将所有切点配置统一规范为本系统的业务包路径，保证了 Spring 容器的正常启动与切面切点的正确植入。

第二是关于本地端口资源冲突与前端端口自动漂移的解决。在本地开发调试阶段，由于本地腾讯 QQ 进程占用了默认的 8083 端口，导致业务微服务无法正常绑定监听，控制台抛出端口占用异常。团队通过代码重构，将业务微服务迁移至 8088 端口，并同步更新了 AOP 切点拦截范围与各跨服务调用的接口配置。同时，在运行 Playwright 端端测试时，前端预览服务在端口冲突时会自动发生端口漂移，导致自动化测试脚本中硬编码的预览地址失效。我们通过在前端预览配置文件中将预览端口固定为 8087 并开启 strictPort 强制锁定选项，使得 Vite 遇到端口占用时直接报错退出而不是漂移，从而确保了自动化测试路径的绝对一致与可靠。

第三是关于安全过滤链排除与监控端点白名单适配的解决。支付和通知微服务在集成监控模块后，因为继承了若依默认的安全配置，导致外部测试工具发起的 Mock 支付请求、WebSocket 连接以及 Actuator 监控拉取被 Spring Security 拦截并返回 401 未授权错误。团队的解决策略是在两个子服务的启动类上通过 Exclude 属性排除了安全与监控安全自动配置类，完全移除了其安全过滤链拦截。对于必须保留安全拦截的管理服务，我们则在 SecurityConfig 配置类中将监控路径添加至匿名白名单，从而打通了本地联调通道与外部监控拉取的连接，保障了测试的顺畅执行。

第四是关于微信异步回调下的 Feign 空上下文及类型转换崩溃的解决。在支付成功异步回调阶段，由于该 HTTP 请求属于微信服务发起，不携带当前登录用户的 JWT 令牌上下文，这导致经过网关和 OpenFeign 转发时因缺少认证头而频繁抛出鉴权失败异常。同时，在订单状态更新接口中，传入的订单号由于带有特定前缀，导致直接调用 Long.valueOf 方法将其转为主键 ID 时触发了 Java 运行时的数字格式化异常。我们通过在业务层引入正则表达式校验，仅对全数字订单号调用转换方法，对含有非数字字符的订单号降级为根据订单字段的多条件查询；同时我们在内部通信的安全策略上进行了局部放行，成功解决了这一跨微服务状态流转过程中的深层异常。

6.3 系统局限性与不足

尽管系统当前作为大作业设计已经完全达成了各项技术指标和测试要求，但在应对真正生产级高并发、高可用环境的标准时，依然存在两点主要的局限性。

首先是单物理数据库分表隔离带来的局限。为简化本地演示的部署门槛和数据初始化流程，当前各个微服务模块依然共享同一个物理 MySQL 数据库实例，只是通过表前缀进行了逻辑划分。这种共享物理数据源的设计，无法模拟出真实的跨网络物理数据库隔离场景，也使得服务之间的解耦不够彻底。在真正的分布式环境下，每个微服务应拥有完全独立的数据库实例。当前的单库设计避开了分布式事务和分布式锁的复杂场景，对于跨微服务的强一致性数据操作，未能在底层数据库层面完全暴露出分布式一致性问题（例如多物理库提交失败时的数据回滚），这是当前系统最大的局限所在。

其次是缺乏完整的分布式链路追踪与系统级限流手段。为了防止本地开发机器因内存资源不足而导致系统卡顿甚至蓝屏，我们在微服务集成时暂时移除了 Sentinel 限流降级服务以及 SkyWalking 链路追踪服务。这导致系统在面临高并发流量冲击时缺乏流量塑形和网关防刷能力，无法在网关层对恶意流量进行阻断。同时，由于缺乏 SkyWalking 的 APM 监控，微服务之间的多级 Feign 调用链路对运维人员来说是一个黑盒，若某个调用环节发生响应慢、数据库死锁或网络丢包，运维团队无法在可视化大屏上通过调用拓扑图和链路追踪数据对慢 SQL 或故障节点进行快速定位，给后期的系统级调优和维护带来不便。

6.4 后续展望

在未来的系统迭代和技术演进中，团队计划从容器化部署、分布式事务控制以及系统监控治理三个方向对生鲜系统进行深度拓展。

第一是全面实现云原生的容器化部署。后续团队将在 `deploy` 分支中编写全套 Dockerfile，将管理后台前端应用、五个后端微服务 JAR 包打包为轻量级的 Docker 镜像。同时编写 `docker-compose.yml` 配置文件，将 Nginx 反向代理、Nacos 服务集群、Redis 哨兵高可用实例、RabbitMQ 镜像队列以及 MySQL 主从复制实例整合在统一的虚拟容器网络中。这不仅能够真正免去演示机器对本地安装各类中间件的依赖，还能通过一条命令实现一键拉起与弹性扩缩容，真正达到云原生就绪状态。

第二是引入分布式事务协调器以保障数据最终一致性。针对微服务拆分后涉及多物理数据库的写入场景，团队计划在系统中集成阿里开源的 Seata 分布式事务组件。在核心业务服务与支付服务涉及数据变更的方法上，利用 Seata 提供的全局

事务注解，将远程 **Feign** 调用和本地 **Mapper** 写入包装在同一个全局事务中。通过 **AT**（自动事务）模式或 **TCC**（补偿型事务）模式，在网络抖动或下游服务抛出异常时实现全局数据回滚，从而在微服务多库物理隔离后依然能保障订单与支付状态的强一致性。

第三是搭建基于 **Prometheus** 与 **Grafana** 的中心化监控体系。针对目前缺乏链路观测的不足，计划在各微服务中集成 **Actuator** 的 **Prometheus** 适配包，暴露标准的指标数据采集端点。在容器网络中部署 **Prometheus** 时序数据库实例，使其每隔十秒自动拉取各微服务实例的 **JVM** 内存使用、**CPU** 负载、垃圾回收耗时以及 **Active** 线程数。最终在 **Grafana** 中设计搭建多维度的图形可视化大屏，全天候、直观地监控微服务集群的运行状况，提升微服务集群的整体可观测性与运维敏捷度。

致谢

在本次分布式应用技术课程大作业完成过程中，感谢指导教师江亮老师在课程要求、技术方向和项目规范方面给予的指导。通过本次项目，我们进一步理解了 **Spring Boot** 后端开发、云端数据库、接口联调、前后端分离和推荐系统设计之间的关系。

同时，感谢小组成员在资料整理、数据库检查、接口测试、算法设计和报告撰写中的协作。本项目从封面格式、章节结构到系统架构图、推荐流程图和数据库说明均按照课程模板进行整理，使最终报告能够较完整地呈现项目设计过程。

最后，感谢开源社区提供的 **Spring Boot**、**React**、**PostgreSQL**、**Redis**、**Supabase** 和 **Apifox** 等工具与技术资料。这些工具降低了分布式应用原型设计和课程展示的实现成本，也为后续继续完善项目提供了基础。

参考文献

- [1] VMware. Spring boot reference documentation[EB/OL]. 2026. <https://docs.spring.io/spring-boot/>.
- [2] Supabase. Supabase documentation[EB/OL]. 2026. <https://supabase.com/docs>.
- [3] PostgreSQL Global Development Group. Postgresql documentation[EB/OL]. 2026. <https://www.postgresql.org/docs/>.
- [4] Redis. Redis documentation[EB/OL]. 2026. <https://redis.io/docs/>.
- [5] Apifox. Apifox API 协作平台文档[EB/OL]. 2026. <https://docs.apifox.com/>.
- [6] Ricci F, Rokach L, Shapira B. Recommender systems handbook[M]. New York: Springer, 2022.
- [7] Covington P, Adams J, Sargin E. Deep neural networks for youtube recommendations[C]//Proceedings of the 10th ACM Conference on Recommender Systems. 2016: 191-198.
- [8] Cheng H T, Koc L, Harmsen J, et al. Wide & deep learning for recommender systems[C]//Proceedings of the 1st Workshop on Deep Learning for Recommender Systems. 2016: 7-10.
- [9] Hu Y, Koren Y, Volinsky C. Collaborative filtering for implicit feedback datasets[C]//Proceedings of the 8th IEEE International Conference on Data Mining. 2008: 263-272.
- [10] Rendle S, Freudenthaler C, Gantner Z, et al. Bpr: Bayesian personalized ranking from implicit feedback[C]//Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence. 2009: 452-461.
- [11] Carbonell J, Goldstein J. The use of mmr, diversity-based reranking for reordering documents and producing summaries[C]//Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. 1998: 335-336.
- [12] He X, Liao L, Zhang H, et al. Neural collaborative filtering[C]//Proceedings of the 26th International Conference on World Wide Web. 2017: 173-182.